

Section 4: Assembling and Deploying Applications (22%) – MCQ Practice Set

60 Questions | Difficulty: Beginner → Proficiency | Exam: Databricks Certified Generative AI Engineer Associate

Question 1

Difficulty: Beginner

A developer wants to wrap a custom Python inference function — including input validation and output formatting — as a deployable MLflow model. Which MLflow class should they subclass?

- **A)** `mlflow.sklearn.SklearnModel` — because all Python-based ML models in Databricks must extend the scikit-learn base class to enable automatic serialization and deserialization during serving.
- **B)** `mlflow.pyfunc.PythonModel` — because it is MLflow's generic Python function wrapper that lets you implement any custom pre-processing, model call, and post-processing logic inside `load_context()` and `predict()` methods.
- **C)** `mlflow.langchain.LangChainModel` — because Databricks requires all custom Python chains to inherit from the LangChain model base class to ensure compatibility with the Model Serving endpoint runtime.
- **D)** `mlflow.models.BaseModel` — because all custom MLflow models must extend `BaseModel` as the root class, with specialized flavors like pyfunc or sklearn registered as plugins that extend this root class.

Correct Answer: B **Explanation:** B is correct. `mlflow.pyfunc.PythonModel` is MLflow's generic "Python function" base class for creating fully custom model wrappers. You subclass it and implement `load_context()` (for one-time initialization at endpoint startup) and `predict()` (for per-request inference with any custom pre/post-processing logic). It is the correct tool when standard framework flavors (LangChain, sklearn) don't provide sufficient control. A is wrong because `mlflow.sklearn` is for scikit-learn estimators implementing `fit()` / `predict()` — custom Python chains are not scikit-learn estimators and cannot be logged with this flavor. C is wrong because there is no `mlflow.langchain.LangChainModel` base class to subclass — LangChain models are logged using `mlflow.langchain.log_model()` directly, not via subclassing. D is wrong because `mlflow.models.BaseModel` does not exist as a user-subclassable class in MLflow's public API — the correct base class for custom Python models is `mlflow.pyfunc.PythonModel`. **Source:** Section 4: Assembling and Deploying – Objective 1: Code a chain using a pyfunc model with pre- and post-processing — docs.databricks.com (search: "mlflow.pyfunc PythonModel custom")

Question 2

Difficulty: Beginner

A production Model Serving endpoint was created under a developer's personal user account. The developer leaves the company, and their account is deactivated. What happens to the endpoint's access to Unity Catalog resources?

- **A)** The endpoint continues to operate normally because Databricks Model Serving endpoints use anonymous compute identities that are independent of the creator's user account after deployment.
- **B)** The endpoint loses access to Unity Catalog resources (Delta tables, UC Functions, Vector Search indexes) because the endpoint's data access identity is permanently bound to the creator's account, which is now inactive.
- **C)** The endpoint automatically migrates to the workspace admin's identity, inheriting the admin's Unity Catalog permissions and continuing to operate with full access to all resources.
- **D)** The endpoint enters a read-only mode, allowing existing cached embeddings to be served but blocking any new Delta table reads or Vector Search queries until a new creator account is assigned.

Correct Answer: B **Explanation:** B is correct. In Databricks Model Serving, the endpoint's Unity Catalog data access identity is permanently bound to the identity of the user who CREATED the endpoint. When that user's account is deactivated, the endpoint can no longer authenticate to Unity Catalog resources (Delta tables, UC Functions, Vector Search indexes) — causing runtime errors during inference. This is exactly why Databricks best practices mandate creating production endpoints under a dedicated Service Principal (a non-human machine account) rather than a personal user account — Service Principals are not tied to individual employees. A is wrong because Model Serving endpoints are not anonymous — they use the creator's identity for all data access operations. C is wrong because Databricks does not automatically migrate the endpoint's identity to a workspace admin — identity migration requires explicit reconfiguration. D is wrong because there is no "read-only mode" in Model Serving — the endpoint either succeeds or fails its data access calls based on the creator's permissions.

Source: Section 4: Assembling and Deploying – Objective 2: Control access to resources from model serving endpoints — docs.databricks.com (search: "Model serving endpoint permissions Databricks")

Question 3

Difficulty: Beginner

A developer builds a simple RAG chain using LangChain: `chain = prompt | llm | output_parser`. They want to log it to MLflow for deployment. Which logging function is most appropriate for this standard LangChain chain?

- **A)** `mlflow.pyfunc.log_model(python_model=CustomClass())` — because all Databricks deployable chains must be wrapped in a custom `PythonModel` class to provide the serving runtime with the necessary metadata.
- **B)** `mlflow.langchain.log_model(lc_model=chain, artifact_path="chain", input_example={...})` — because it natively serializes LangChain chains (including LCEL runnables) with automatic dependency detection and signature inference.
- **C)** `mlflow.sklearn.log_model(sk_model=chain, artifact_path="chain")` — because LangChain's `|` pipe operator is equivalent to scikit-learn's `Pipeline`, making the sklearn flavor compatible with LangChain chains.
- **D)** `mlflow.spark.log_model(spark_model=chain, artifact_path="chain")` — because LangChain chains run on Spark executors in Databricks and must be serialized with the Spark MLflow flavor to ensure distributed execution compatibility.

Correct Answer: B **Explanation:** B is correct. `mlflow.langchain.log_model()` is the designated MLflow flavor for logging LangChain chains and LCEL runnables. It handles automatic serialization of the chain, detects Python package dependencies, and infers the model signature from the `input_example`. This is the standard, simplest approach for deploying a standard LangChain chain — no custom wrapping required. A is wrong because `mlflow.pyfunc.log_model()` with a custom `PythonModel` class is for complex custom logic that goes beyond standard LangChain chains — using it for a simple `prompt | llm | parser` chain adds unnecessary boilerplate. C is wrong because scikit-learn's `Pipeline` and LangChain's `|` LCEL operator are unrelated — LangChain chains implement different interfaces and cannot be serialized with the sklearn flavor. D is wrong because LangChain chains run on the driver node as Python code, not on Spark executors as distributed ML models; the Spark MLflow flavor is for Spark ML pipelines, not LangChain. **Source:** Section 4: Assembling and Deploying – Objective 3: Code a simple chain according to requirements — [docs.databricks.com](https://docs.databricks.com/search/Log+and+load+LangChain+models+MLflow+Databricks) (search: "Log and load LangChain models MLflow Databricks")

Question 4

Difficulty: Beginner

When registering an MLflow model to Unity Catalog, what naming convention must the model name follow?

- **A)** The model name must be a simple string with no dots or slashes (e.g., `rag_app_v2`), and Unity Catalog automatically assigns it to the default catalog and schema of the current workspace.
- **B)** The model name must follow the three-level namespace `catalog.schema.model_name` (e.g., `main.my_schema.rag_app`) to specify exactly which Unity Catalog location owns and governs the model.
- **C)** The model name must follow the format `workspace_id/experiment_id/model_name` to link the Unity Catalog model to its originating MLflow experiment run for full lineage tracking.
- **D)** The model name must be prefixed with `uc://` (e.g., `uc://main.my_schema.rag_app`) to signal to MLflow that the target registry is Unity Catalog rather than the default MLflow Model Registry.

Correct Answer: B **Explanation:** B is correct. Unity Catalog uses a three-level namespace for all objects:

`catalog.schema.object_name`. When registering a model to Unity Catalog, the `registered_model_name` parameter must follow this convention (e.g., `"main.my_schema.rag_app"`). This determines which catalog and schema governs the model — controlling permissions, lineage tracking, and discoverability. A is wrong because a flat name without dots would attempt to register to the legacy MLflow Model Registry (if `mlflow.set_registry_uri("databricks-uc")` is not set), not to Unity Catalog; Unity Catalog always requires the three-level namespace. C is wrong because the workspace/experiment IDs are not part of the model registration name — MLflow automatically tracks lineage from the originating run; the model name is just the Unity Catalog three-level reference. D is wrong because there is no `uc://` prefix convention in MLflow model registration — the Unity Catalog registry is selected by calling `mlflow.set_registry_uri("databricks-uc")` before registration, not by prefixing the model name. **Source:** Section 4: Assembling and Deploying – Objective 5: Register the model to Unity Catalog using MLflow — docs.databricks.com (search: "Register model Unity Catalog MLflow")

Question 5

Difficulty: Beginner

What is the purpose of the `load_context()` method in an `mlflow.pyfunc.PythonModel` subclass?

- **A)** `load_context()` is called on every inference request to reload the model's context (retrieved documents, conversation history) from the latest state in the database, ensuring each prediction uses fresh data.
- **B)** `load_context()` is called once when the Model Serving endpoint starts up, and is used to load heavy objects like LLM clients, tokenizers, or configuration files that should be initialized once and reused across all requests.
- **C)** `load_context()` loads the MLflow model's input example and signature from the artifact store and validates that each incoming request matches the expected schema before passing it to `predict()`.
- **D)** `load_context()` registers the model's dependencies with the Databricks serving runtime at startup, downloading and installing any Python packages listed in `pip_requirements` that are not already available in the serving environment.

Correct Answer: B **Explanation:** B is correct. `load_context()` runs exactly once when the Model Serving endpoint initializes (cold start). Its purpose is to perform expensive one-time initialization — creating LLM API clients, loading tokenizers, reading configuration files, connecting to Vector Search — so these resources are ready and reused across all subsequent requests without being re-initialized per request. This dramatically reduces per-request latency. A is wrong because `load_context()` is NOT called on every request — it is called only once at startup. Reloading context per-request would be the correct description of accessing session state within `predict()`. C is wrong because signature and input example validation is handled by MLflow's serving framework automatically — `load_context()` is for user-defined initialization code, not MLflow's internal validation logic. D is wrong because Python package installation from `pip_requirements` is handled by the MLflow serving runtime during environment setup, not within `load_context()` — the method runs after the environment is already set up. **Source:** Section 4: Assembling and Deploying – Objective 1: Code a chain using a pyfunc model — docs.databricks.com (search: "mlflow.pyfunc PythonModel custom")

Question 6

Difficulty: Beginner

A developer calls `vsc.create_delta_sync_index()` with `pipeline_type="TRIGGERED"`. Three hours later, 500 new document chunks are appended to the source Delta table. Will those new chunks be searchable in the Vector Search index immediately?

- **A)** Yes — Delta Sync indexes with `TRIGGERED` mode monitor the source Delta table's transaction log in real time and automatically index new rows within 30 seconds of the APPEND operation completing.
- **B)** No — `TRIGGERED` mode only syncs when explicitly triggered by calling `index.sync()` via the SDK or REST API. Until a sync is triggered, the 500 new chunks are invisible to the Vector Search index.
- **C)** Yes — Delta Lake's ACID transaction guarantees mean that any committed row in the source Delta table is immediately queryable through the Vector Search index, regardless of the sync pipeline type configuration.
- **D)** No — new rows are only indexed during the nightly Databricks Vector Search maintenance window that runs at 2:00 AM UTC, regardless of whether `TRIGGERED` or `CONTINUOUS` sync mode is configured.

Correct Answer: B **Explanation:** B is correct. `TRIGGERED` pipeline mode means the Vector Search index does NOT automatically detect and propagate changes from the source Delta table. It only updates when a sync is explicitly initiated — either by calling `index.sync()` programmatically via the Python SDK, via the Databricks REST API, or through the Databricks UI's "Sync now" button. Until that manual trigger fires, the 500 new chunks exist in the Delta table but are completely invisible to the Vector Search index. A is wrong because real-time automatic monitoring is the behavior of `CONTINUOUS` mode, not `TRIGGERED` mode — confusing these two modes is a common exam trap. C is wrong because Delta ACID guarantees apply to the source Delta table's consistency, not to the derived Vector Search index — the index is a separate derived artifact that must be explicitly synchronized. D is wrong because there is no automatic nightly maintenance window for Vector Search sync — all syncs in `TRIGGERED` mode are explicitly user-initiated. **Source:** Section 4: Assembling and Deploying – Objective 6: Create and query a Vector Search index — [docs.databricks.com](https://docs.databricks.com/search/create-mosaic-ai-vector-search-index) (search: "Create Mosaic AI Vector Search index")

Question 7

Difficulty: Beginner

What is the key difference between Databricks Foundation Model API's "Pay-per-token" mode and "Provisioned Throughput" mode?

- **A)** Pay-per-token uses cloud GPU instances managed by the customer, while Provisioned Throughput uses Databricks-managed serverless GPUs — the difference is who manages the underlying hardware infrastructure.
- **B)** Pay-per-token is serverless (no capacity reservation) with per-token billing — best for prototyping and variable workloads. Provisioned Throughput reserves a guaranteed tokens-per-second capacity — required for production SLAs, compliance, and fine-tuned models.
- **C)** Pay-per-token only supports Llama-3 family models, while Provisioned Throughput supports any open-source model including Mistral, Mixtral, and CodeLlama, giving broader model selection flexibility.
- **D)** Pay-per-token charges for both input and output tokens, while Provisioned Throughput charges only for output tokens since the input is processed by a shared embedding cache that amortizes input token costs.

Correct Answer: B **Explanation:** B is correct. The fundamental difference is capacity management: Pay-per-token is a shared, serverless pool with no capacity reservation — you pay for exactly what you use, there's no latency guarantee, and it may throttle under high load. This makes it ideal for development and low-volume prototyping. Provisioned Throughput reserves a dedicated number of tokens-per-second (TPS), guaranteeing that capacity regardless of other traffic — essential for production applications with latency SLAs, for HIPAA-compliant workloads requiring dedicated compute, and for deploying custom fine-tuned models. A is wrong because both modes run on Databricks-managed infrastructure — the customer doesn't manage GPUs in either case; the difference is capacity reservation, not infrastructure ownership. C is wrong because Pay-per-token supports multiple model families (Llama, Mistral, Mixtral, DBRX), not just Llama-3; and Provisioned Throughput also supports these models plus custom fine-tuned versions. D is wrong because both modes charge for input and output tokens — there is no "input caching" pricing mechanism that eliminates input token charges for Provisioned Throughput. **Source:** Section 4: Assembling and Deploying – Objective 7: Identify how to serve an LLM application that leverages Foundation Model APIs — docs.databricks.com (search: "Databricks Foundation Model APIs")

Question 8

Difficulty: Beginner

A team wants to run sentiment classification on 5 million customer reviews stored in a Delta table as a nightly batch job. Which Databricks tool is most appropriate?

- **A)** LangChain with a `ChatDatabricks` LLM and a Python `for` loop that iterates over each review row in a Spark DataFrame, calling the LLM API once per row and writing results back to a new Delta column.
- **B)** A LangGraph agent with a classification node that processes each review as a separate conversation turn, maintaining sentiment state across the full 5 million review dataset.
- **C)** Databricks `ai_query()` SQL function in a serverless SQL warehouse query that selects all 5 million rows and applies the classification in a single SQL statement, letting Databricks parallelize across executors automatically.
- **D)** A Databricks Workflow with 5 million individual Tasks — one per review — each calling a Foundation Model API REST endpoint, scheduled to complete before the morning business hours begin.

Correct Answer: C **Explanation:** C is correct. `ai_query()` is Databricks' batch inference tool: it allows you to apply a model to every row of a Delta table in a single SQL query. The serverless SQL warehouse automatically parallelizes the inference calls across its executors, handles rate-limit retries, and processes millions of rows efficiently without any custom orchestration code. A is wrong because a Python `for` loop over 5 million rows is sequential — it processes one review at a time, would take hours or days, and misses all of Databricks' native parallelization capabilities. B is wrong because LangGraph is for stateful, multi-step reasoning agents — using it to classify 5 million independent reviews with no inter-row state adds extreme orchestration overhead for a task that is trivially addressed by `ai_query()`. D is wrong because creating 5 million Workflow Tasks is computationally absurd — Databricks Workflows are designed for dozens to hundreds of tasks representing pipeline stages, not one task per data row. **Source:** Section 4: Assembling and Deploying – Objective 9: Identify batch inference workloads and apply `ai_query()` — docs.databricks.com (search: "ai_query function Databricks SQL")

Question 9

Difficulty: Beginner

A team uses the MLflow Prompt Registry to manage their RAG system prompt. They have the prompt registered as version 1 (dev), version 2 (staging), and version 3 (production). The application code loads the prompt using `mlflow.load_prompt("prompts:/support_prompt/production")`. What does "promoting" the prompt to a new version involve?

- **A)** Rewriting the application code to change the version number from `"production"` to a new integer (e.g., `"prompts:/support_prompt/4"`), then redeploying the serving endpoint with the updated code pointing to version 4.
- **B)** Re-assigning the `production` alias in the MLflow Prompt Registry to point to the new (validated) prompt version — the application code remains unchanged because it loads by alias (`"production"`), not by version number.
- **C)** Creating a new MLflow experiment run with the new prompt version as a run parameter, then updating the model serving endpoint to load from the latest experiment run ID rather than the prompt registry alias.
- **D)** Deleting the current `production` alias from version 3 in the Registry, creating a new `production_v4` alias on the new version, and updating the application code to reference the new alias name.

Correct Answer: B **Explanation:** B is correct. This is the core benefit of prompt aliases in the MLflow Prompt Registry. The application code references the prompt by a mutable alias (`"production"`) — not by an immutable version number. When a new prompt version is validated and ready for production, the team re-assigns the `production` alias to point to the new version. Because the application code always loads `"prompts:/support_prompt/production"`, it automatically picks up the newly aliased version WITHOUT any code change or redeployment. This is the prompt equivalent of a blue-green deployment — zero-downtime promotion with full version history preserved. A is wrong because changing the version number in application code requires a code change and redeployment — this defeats the purpose of using aliases (which eliminate the need for code changes during promotion). C is wrong because experiment run IDs are for model and metric tracking, not for prompt version promotion — the Prompt Registry's alias system handles this more cleanly. D is wrong because creating a new alias name (`production_v4`) still requires updating the application code to reference the new alias — this negates the alias benefit. The correct pattern is to reuse the same `production` alias name while updating which version it points to. **Source:** Section 4: Assembling and Deploying – Objective 14: Apply prompt version control and manage prompt lifecycle — docs.databricks.com (search: "MLflow Prompt Registry Databricks")

Question 10

Difficulty: Beginner

An internal analytics team wants business users to query their company's sales data using natural language from within Microsoft Teams. What is the recommended Databricks interface?

- **A)** Deploy a custom Streamlit web app on Databricks Apps and send users the Databricks Apps URL, which they can open in a Teams tab via the Teams website hosting feature.
- **B)** Use the Databricks Genie app for Microsoft Teams — admin installs it from the Microsoft marketplace, and business users interact with Genie Agents by mentioning `@Databricks Genie` directly in Teams channels.
- **C)** Deploy a Databricks Model Serving endpoint and instruct the business users to send HTTP POST requests to the endpoint URL from the Teams chat box using the `/request` slash command.
- **D)** Use Databricks Workflows to schedule a daily SQL report that is exported to a SharePoint folder, which Teams syncs as a channel notification — providing users with the latest data through their Teams interface.

Correct Answer: B **Explanation:** B is correct. The Databricks Genie app for Microsoft Teams is the purpose-built integration for business users who live in Teams. A workspace admin installs the Genie app from the Microsoft App Marketplace (or through Microsoft Teams admin center), and users interact by mentioning `@Databricks Genie` in any Teams channel or chat. They can ask natural language questions about data, and the Genie Agent generates and executes the appropriate SQL queries, returning results directly in Teams. A is wrong because while Databricks Apps can be hosted as a web app, embedding a Streamlit app in Teams requires manual Teams Tab configuration and doesn't provide the seamless `@Genie` chat experience that business users expect. C is wrong because business users (non-technical) cannot be expected to craft HTTP POST requests from a chat interface — this is a developer-facing API pattern, not a business user interface. D is wrong because scheduled daily SQL reports are batch outputs, not interactive natural language querying — users cannot ask follow-up questions or explore data dynamically through a daily report export. **Source:** Section 4: Assembling and Deploying – Objective 15: Develop an appropriate interactive user-facing interface — docs.databricks.com (search: "Databricks Genie app Slack Teams")

Question 11

Difficulty: Intermediate

A developer stores an OpenAI API key required by their model serving endpoint. What is the Databricks-secure implementation?

- **A)** Hardcode the OpenAI API key as a Python string constant in the model's `load_context()` method — since the source code is stored in a private Git repository with restricted access, the key is protected by Git repository access controls.
- **B)** Store the key in a Databricks Secret Scope, reference it as an environment variable in the endpoint's serving configuration (e.g., `OPENAI_API_KEY`), and read it in `load_context()` via `os.environ["OPENAI_API_KEY"]`.
- **C)** Store the key in a Unity Catalog Delta table row with column-level encryption enabled, and have the `predict()` method query this table on every request to retrieve the latest key value before each LLM call.
- **D)** Pass the OpenAI API key as a parameter in the JSON inference request payload from the calling application, so each caller provides their own key and the endpoint never needs to store credentials internally.

Correct Answer: B **Explanation:** B is correct. The Databricks-secure pattern for external API credentials is: (1) Store the key in a Databricks Secret Scope (an encrypted, access-controlled secret store). (2) Reference the secret as an environment variable in the Model Serving endpoint configuration — Databricks injects the secret's value as an environment variable at endpoint startup. (3) Read the environment variable in `load_context()` using `os.environ["OPENAI_API_KEY"]`. This pattern ensures the key is never in plain text in code or logs. A is wrong because hardcoding API keys in source code (even private repos) is a critical security anti-pattern — keys in source code can be leaked through git history, accidentally committed to public repos, or exposed in logs and error messages. C is wrong because querying a Delta table for the API key on every request adds latency to each inference call and is operationally complex — Databricks Secrets are the purpose-built secure credential store, not Delta tables. D is wrong because passing API keys in request payloads is a serious security exposure — they appear in network logs, Inference Table records, and could be intercepted in transit; each caller also needs to know the key. **Source:** Section 4: Assembling and Deploying – Objective 2: Control access to resources from model serving endpoints — docs.databricks.com (search: "Databricks Secrets model serving")

Question 12

Difficulty: Intermediate

A developer calls `mlflow.langchain.log_model()` without providing an `input_example`. They then try to register the model to Unity Catalog with `mlflow.register_model()`. What happens?

- **A)** The registration succeeds normally — Unity Catalog model registration does not require a model signature; it only requires a valid MLflow run URI and a three-level namespace model name.
- **B)** The registration fails with an error stating that a model signature is required for Unity Catalog registration — without an `input_example` to auto-infer the signature, the developer must manually define and attach a signature using `mlflow.models.infer_signature()`.
- **C)** The registration succeeds but the serving endpoint created from this model will not accept any JSON payloads — it only accepts binary Parquet data because the missing signature defaults to a binary input schema.
- **D)** The registration succeeds but MLflow assigns a default signature of `{inputs: string, outputs: string}` — the developer must update this in the Unity Catalog UI after registration before deploying to a serving endpoint.

Correct Answer: B **Explanation:** B is correct. Unity Catalog requires a model signature for all registered models. The model signature defines the expected input/output schema (data types and column names) — Unity Catalog uses it to validate inference API payloads and to display the model's interface in Catalog Explorer. The easiest way to include a signature is to provide an `input_example` to `log_model()`, which causes MLflow to automatically infer the signature. Without either an `input_example` OR a manually defined signature, the `register_model()` call fails with a signature requirement error. A is wrong because Unity Catalog DOES require a model signature — this is a hard requirement for UC-registered models, unlike the legacy MLflow Model Registry which allowed signature-less models. C is wrong because there is no "defaults to binary Parquet" behavior — missing signatures cause a registration error, not a format downgrade. D is wrong because MLflow does not auto-assign a default `{string, string}` signature — missing signatures result in a registration failure, not a fallback schema. **Source:** Section 4: Assembling and Deploying – Objective 5: Register the model to Unity Catalog using MLflow — docs.databricks.com (search: "Register model Unity Catalog MLflow" and "MLflow model signature input example")

Question 13

Difficulty: Intermediate

A developer creates a Vector Search Delta Sync index using Databricks-managed embeddings. When a user submits a search query (`index.similarity_search(query_text="refund policy", num_results=5)`), what happens internally to the query text before the vector search is performed?

- **A)** The query text is sent directly to the Vector Search index as a raw string — the index performs keyword matching (BM25) against the stored chunk text fields in the source Delta table, not against embedding vectors.
- **B)** The query text is passed to the same embedding model endpoint specified during index creation (e.g., `databricks-bge-large-en`), which converts it to a vector — that query vector is then compared against the stored chunk embeddings using approximate nearest neighbor search.
- **C)** The query text is first tokenized and chunked using the same chunking strategy applied to the source documents, then each query chunk is separately embedded and the results are aggregated across all query chunk matches.
- **D)** The query text is sent to a Databricks-hosted cross-encoder model that scores it directly against all stored chunk texts in the Delta table, returning a ranked list without any embedding computation.

Correct Answer: B Explanation: B is correct. When `similarity_search()` is called with `query_text` , the Vector Search service automatically sends the query text to the same embedding model endpoint that was specified in `embedding_model_endpoint_name` during index creation (e.g., `"databricks-bge-large-en"`). The embedding model converts the query text into a vector, and that query vector is compared against all stored chunk embeddings using an approximate nearest neighbor (ANN) algorithm (e.g., HNSW). The `num_results` most similar embeddings are returned. This guarantees that query and document embeddings are in the same vector space. A is wrong because Mosaic AI Vector Search is a semantic (dense vector) search system, not a keyword (BM25) search system — it compares embedding vectors, not raw text strings. C is wrong because user queries are not chunked before embedding — queries are typically short (a sentence or phrase) and are embedded as a single vector; query chunking would fragment the query's semantic meaning. D is wrong because cross-encoder reranking is an optional second-stage component added AFTER vector search returns candidates — it is not the primary search mechanism used by default in Vector Search. **Source:** Section 4: Assembling and Deploying – Objective 6 & 8: Create/query Vector Search and key concepts — docs.databricks.com (search: "Query Vector Search index Databricks")

Question 14

Difficulty: Intermediate

A company needs to run `ai_query()` for batch inference on 10 million rows. The data engineer attempts to run it on a Pro SQL Warehouse but gets an error. What is the correct compute requirement?

- **A)** `ai_query()` requires a Classic SQL Warehouse with at least 4 worker nodes — Pro warehouses lack the distributed execution framework needed for batch AI inference workloads.
- **B)** `ai_query()` requires a Serverless SQL Warehouse — it does not work on Pro or Classic warehouses because it relies on serverless compute's ability to dynamically scale and manage AI function execution.
- **C)** `ai_query()` requires a Databricks cluster with Databricks Runtime ML 13.0+, not a SQL Warehouse — batch AI inference is a cluster-based operation, not a SQL warehouse operation.
- **D)** `ai_query()` requires a Photon-enabled Pro Warehouse with at least 16 DBUs reserved — Photon acceleration is mandatory for AI function inference to achieve acceptable throughput on million-row datasets.

Correct Answer: B **Explanation:** B is correct. `ai_query()` (and other SQL AI functions) specifically require a Serverless SQL Warehouse to execute. Pro and Classic SQL Warehouses do not support `ai_query()` — attempting to run it on these warehouse types results in an error. The Serverless SQL Warehouse provides the auto-scaling, managed infrastructure and the integrated AI function runtime needed for efficient batch inference. Databricks Runtime 18.2+ is also a requirement for some AI function features. A is wrong because Classic SQL Warehouses do not support `ai_query()` — the requirement is Serverless, not Classic. C is wrong because `ai_query()` is a SQL function designed for SQL Warehouses — not a cluster-based PySpark operation; cluster-based batch inference would use UDFs or `applyInPandas`. D is wrong because Photon is a vectorized query engine for analytical SQL — it does not enable `ai_query()` functionality, and Photon is not a requirement for AI functions. **Source:** Section 4: Assembling and Deploying – Objective 9: Identify batch inference workloads and apply `ai_query()` — docs.databricks.com (search: "ai_query function Databricks SQL" and "Batch inference AI functions")

Question 15

Difficulty: Intermediate

A developer builds a stateful customer service agent using LangGraph. The agent must remember the user's name and preferred language from previous sessions (cross-session memory), but also maintain the current conversation history within a single session (short-term memory). Which Databricks components handle each requirement?

- **A)** Both short-term and long-term memory are handled by Databricks Inference Tables — the table captures all past inputs and outputs and the agent queries it at the start of each session to reconstruct the conversation context.
- **B)** Short-term (within-session) memory is handled by LangGraph's Checkpointer (writing conversation turns to Lakebase using a thread ID), and long-term (cross-session) memory is handled by LangGraph's Store API reading/writing user preferences to Lakebase tables.
- **C)** Both short-term and long-term memory are handled by Databricks Secrets — user names and preferences are stored as secret key-value pairs that the agent reads at the start of each session to reconstruct the user's context.
- **D)** Short-term memory is handled by the LLM's internal attention mechanism (KV cache), and long-term memory is handled by Unity Catalog Delta tables that the agent explicitly queries using `ai_query()` at each session start.

Correct Answer: B **Explanation:** B is correct. LangGraph provides two complementary memory mechanisms on Databricks: (1) **Short-term (within-session) memory** — LangGraph's Checkpointer persists the conversation state (all messages in the current thread) using a thread ID. The Checkpointer writes each new turn to a durable store (e.g., Lakebase Postgres table) so the agent can resume if interrupted. (2) **Long-term (cross-session) memory** — LangGraph's Store API provides a key-value persistence layer where the agent can write and read user-specific facts (name, language preference, past issues) across different sessions using a user ID as the namespace key. Lakebase (managed Postgres) is the recommended durable store for both. A is wrong because Inference Tables are a monitoring tool that logs production traffic for quality analysis — they are not designed or queried as a memory system for active agents. C is wrong because Databricks Secrets store machine credentials (API keys, passwords) — they are not a per-user preference store for agent memory. D is wrong because the LLM's KV cache is a performance optimization for token computation within a single call — it is ephemeral and not accessible as a persistent memory system. **Source:** Section 4: Assembling and Deploying – Objective 11: Configure a persistent datastore to store and retrieve intermediate memory — docs.databricks.com (search: "Agent memory Databricks" and "Lakebase Databricks Postgres")

Question 16

Difficulty: Intermediate

A team wants to use CI/CD for their RAG application with Databricks. They need to ensure the Vector Search index is always consistent with the source code. Which Databricks tool provides Infrastructure-as-Code for defining Vector Search indexes in a version-controlled configuration?

- **A)** Databricks Delta Live Tables (DLT) — the team defines the Vector Search index as a DLT streaming table using the `@dlt.table()` decorator, and the DLT pipeline automatically creates and manages the index configuration.
- **B)** Databricks Asset Bundles (DABs) — the team defines Vector Search endpoints, indexes, and Model Serving endpoints in a `databricks.yml` file that is checked into Git, and deploying the bundle creates or updates these resources consistently across environments.
- **C)** Databricks Workflows — the team creates a Workflow that runs a deployment notebook on every Git push, and the notebook uses the Python SDK to create or update Vector Search indexes in the target environment.
- **D)** MLflow Projects — the team defines the Vector Search index configuration as an MLflow Project entry point in `MLproject`, and `mlflow run` creates the index as part of the model training and deployment pipeline.

Correct Answer: B **Explanation:** B is correct. Databricks Asset Bundles (DABs) is the Infrastructure-as-Code framework for Databricks. You define all infrastructure resources — Vector Search endpoints, Vector Search indexes, Model Serving endpoints, Workflows, and more — in a `databricks.yml` configuration file. This file is stored in Git alongside the application code. Running `databricks bundle deploy` creates or updates these resources in the target Databricks workspace, ensuring infrastructure is always consistent with the codebase. A is wrong because Delta Live Tables are for defining data transformation pipelines (ETL), not infrastructure resources like Vector Search indexes — there is no DLT concept of creating a Vector Search index via `@dlt.table()`. C is wrong because while a deployment notebook in a Workflow can create indexes programmatically, it is not IaC — the configuration is embedded in notebook code, not in a declarative configuration file that can be diffed and version-controlled as infrastructure. D is wrong because MLflow Projects define ML training workflows (parameters, entry points), not infrastructure provisioning — they cannot create Databricks-native resources like Vector Search indexes. **Source:** Section 4: Assembling and Deploying – Objective 12: Apply CI/CD best practices — docs.databricks.com (search: "Databricks Asset Bundles CI/CD")

Question 17

Difficulty: Intermediate

A developer builds a multi-agent system. The Supervisor agent needs to access GitHub repository data (issues, pull requests) via a standard MCP interface. Which type of MCP server should they use?

- **A)** Managed MCP Server — Databricks provides a pre-built managed MCP server for GitHub that is available at zero configuration and requires no registration in Unity Catalog.
- **B)** External MCP Server (MCP Service) — a third-party GitHub MCP server is registered as a Unity Catalog-governed "MCP Service" and connected via Unity AI Gateway, applying Databricks governance to external GitHub data access.
- **C)** Custom MCP Server — the developer must build their own GitHub integration from scratch as a custom MCP server hosted on Databricks Apps, since no standard GitHub MCP server exists.
- **D)** The Genie Agent MCP server — Genie Agents can query any external API including GitHub using natural language, so the Supervisor should connect to the Genie Agent's MCP URL instead of a GitHub-specific server.

Correct Answer: B **Explanation:** B is correct. GitHub integration falls under the "External MCP Server" category — third-party services (GitHub, Slack, Jira, etc.) that have existing MCP server implementations. In Databricks, these are registered as Unity Catalog-governed "MCP Services" and all traffic routes through the Unity AI Gateway for centralized governance, auditability, and security policy enforcement. A is wrong because Databricks Managed MCP Servers only cover Databricks-native resources (Unity Catalog tables/functions, Vector Search indexes, Genie Agents) — GitHub is an external third-party service, not a Databricks-native resource. C is wrong because GitHub has existing open-source MCP server implementations (e.g., the official GitHub MCP server) — the developer does not need to build from scratch; they register the existing server as an External MCP Service. D is wrong because Genie Agents are specialized for natural language → SQL queries against Unity Catalog data tables — they are not general-purpose HTTP/API connectors and cannot query the GitHub API. **Source:** Section 4: Assembling and Deploying – Objective 13: Integrate managed, external, and custom MCP servers — docs.databricks.com (search: "MCP servers Databricks" and "Unity Catalog MCP services")

Question 18

Difficulty: Intermediate

A company's legal team reviews the MLflow Prompt Registry and asks: "If we change the production prompt and something goes wrong, can we see exactly what the prompt looked like before the change?" What does the Prompt Registry provide?

- **A)** No — the MLflow Prompt Registry stores only the current active version of each prompt; previous versions are automatically deleted to conserve storage and keep the registry clean.
- **B)** Yes — every change to a prompt creates an immutable, auto-numbered version snapshot with an optional commit message. The MLflow UI shows a diff between versions, and any previous version can be restored by re-assigning the `production` alias to it.
- **C)** Yes — but only if the team manually exported the prompt template as a JSON file and saved it to a Unity Catalog Volume before making each change; the Prompt Registry itself does not maintain version history.
- **D)** Yes — the Prompt Registry stores the last 5 versions of each prompt automatically. Versions older than 5 changes are permanently deleted, so teams must export older versions to external storage for long-term audit purposes.

Correct Answer: B **Explanation:** B is correct. The MLflow Prompt Registry provides complete, immutable version history for all prompt changes. Key features: (1) Every registered prompt change creates a new auto-numbered version (1, 2, 3...) that is stored permanently and cannot be modified retroactively. (2) Each version can include a commit message documenting why the prompt was changed. (3) The MLflow UI shows diffs between versions. (4) Rolling back to any previous version is as simple as re-assigning the `production` alias to the older version number — no code changes required. A is wrong because the Prompt Registry is explicitly a version CONTROL system — it retains all historical versions, not just the current one; automatic deletion would defeat its entire audit purpose. C is wrong because the Prompt Registry maintains version history natively — no manual export to a Volume is needed; that would be a cumbersome workaround for a feature that is built-in. D is wrong because there is no 5-version limit in the MLflow Prompt Registry — all versions are retained indefinitely until explicitly deleted; the system does not auto-prune old versions. **Source:** Section 4: Assembling and Deploying – Objective 14: Apply prompt version control and manage prompt lifecycle — docs.databricks.com (search: "MLflow Prompt Registry Databricks")

Question 19

Difficulty: Intermediate

A developer wants to expose a custom agent as a web interface for internal users. The agent calls a Databricks Model Serving endpoint. The team has chosen Streamlit for the front-end framework. What is the correct Databricks deployment target for this Streamlit app?

- **A)** Deploy the Streamlit app to Databricks Workflows as a long-running Task, using the Workflow's always-on execution mode to keep the web server running continuously for internal users.
- **B)** Deploy the Streamlit app to Databricks Apps — Apps natively hosts Streamlit (and Gradio, Flask, FastAPI) web applications within the Databricks security perimeter, providing built-in SSO, Unity Catalog identity, and no separate infrastructure.
- **C)** Deploy the Streamlit app to a Databricks job cluster using a Custom Docker Image containing the Streamlit server, with a port forwarding rule in the cluster configuration to expose the web interface.
- **D)** Deploy the Streamlit app to Databricks Delta Live Tables as a continuous streaming pipeline, which runs the Streamlit server as a DLT processing node that handles user HTTP requests as streaming data events.

Correct Answer: B **Explanation:** B is correct. Databricks Apps is the purpose-built service for hosting interactive web applications (Streamlit, Gradio, Flask, FastAPI) within the Databricks ecosystem. Key benefits: the app runs inside the Databricks security perimeter (SSO via Databricks identity), it can access Unity Catalog resources and call Model Serving endpoints using the user's identity, and there is no separate infrastructure to manage — Databricks handles the hosting. A is wrong because Databricks Workflows are for scheduled or triggered batch/streaming data and ML pipeline jobs — they do not support running persistent web servers that accept user HTTP requests. C is wrong because deploying a custom Docker image with port forwarding to a job cluster is a complex, unsupported workaround — job clusters are for computation tasks, not web application hosting; this approach is not supported by Databricks. D is wrong because Delta Live Tables is a declarative data pipeline framework (ETL) — it cannot host a Streamlit web server or handle HTTP requests from end users. **Source:** Section 4: Assembling and Deploying – Objective 15: Develop an appropriate interactive user-facing interface — docs.databricks.com (search: "Databricks Apps deploy agent")

Question 20

Difficulty: Intermediate

A developer needs to choose between a Delta Sync Index and a Direct Vector Access Index for their Vector Search setup. Their use case: they have a third-party system that computes specialized domain-specific embeddings (using a proprietary model not available as a Databricks endpoint) and they need to upload these pre-computed embeddings to the index. Which index type is correct?

- **A)** Delta Sync Index with Databricks-managed embeddings — configure the index to use a Databricks Foundation Model API embedding endpoint and allow Databricks to recompute embeddings from the chunk text, overriding the third-party embeddings.
- **B)** Direct Vector Access Index — this index type allows the developer to upload pre-computed embedding vectors directly, giving full control over the embedding computation without requiring a Databricks-hosted embedding model endpoint.
- **C)** Delta Sync Index with self-managed embeddings — configure the source Delta table to include an embedding column pre-populated with the third-party embeddings, and the Delta Sync index reads the pre-computed values directly from that column.
- **D)** Either index type works identically — both Delta Sync and Direct Vector Access indexes support pre-computed external embeddings through the same `embedding_vector` column specification in the source Delta table.

Correct Answer: C **Explanation:** C is correct — and this is a nuanced choice. A Delta Sync Index with "self-managed embeddings" (where the source Delta table includes a pre-computed embedding column) allows the developer to use ANY embedding model, including proprietary third-party models. The developer computes embeddings externally, stores them as an array column in the Delta table, and the Delta Sync index reads and indexes these pre-computed vectors. This combines the convenience of Delta Sync (automatic CDF-based sync, no manual upload API calls) with the flexibility of external embedding computation. B is also a valid option (Direct Vector Access Index), but C is more appropriate because it leverages the Delta Sync automation (incremental updates via CDF) rather than requiring manual API calls to upload each batch of embeddings. A is wrong because the use case explicitly requires keeping the third-party embeddings — having Databricks recompute them with a different model defeats the purpose. D is wrong because the two index types have fundamentally different APIs and behaviors — they are not identical. **Source:** Section 4: Assembling and Deploying – Objective 8 & 10: Key concepts of Vector Search and configuration — docs.databricks.com (search: "Mosaic AI Vector Search overview" and "Mosaic AI Vector Search configuration")

Question 21

Difficulty: Advanced

A developer creates the following pyfunc model:

```
class CustomChain(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        import openai
        self.client = openai.OpenAI(api_key=os.environ["OPENAI_API_KEY"])
        self.system_prompt = context.artifacts["system_prompt"]

    def predict(self, context, model_input):
        query = model_input["query"].iloc[0]
        response = self.client.chat.completions.create(
            model="gpt-4o",
            messages=[{"role": "system", "content": self.system_prompt},
                      {"role": "user", "content": query}]
        )
        return response.choices[0].message.content
```

They log it with `mlflow.pyfunc.log_model(artifact_path="chain", python_model=CustomChain(), artifacts={"system_prompt": "./system_prompt.txt"})`. What does `context.artifacts["system_prompt"]` return during serving?

- **A)** The string `"./system_prompt.txt"` — the artifacts dictionary stores the local file path as specified at log time, and the serving environment uses this path to locate the file on the serving container's local filesystem.
- **B)** The resolved local file path to the `system_prompt.txt` file as it exists in the serving container (e.g., `/tmp/mlflow-artifacts/system_prompt.txt`) — MLflow copies the artifact to the serving container and provides its local path via the context.
- **C)** The contents of `system_prompt.txt` as a Python string — MLflow reads the file and passes its text content directly through `context.artifacts`.
- **D)** A Spark DataFrame containing the system prompt text — MLflow standardizes all artifact returns to Spark DataFrames for compatibility with Databricks' distributed serving infrastructure.

Correct Answer: B **Explanation:** B is correct. When artifacts are logged with `mlflow.pyfunc.log_model()`, MLflow copies those files into the MLflow artifact store (e.g., DBFS or a cloud storage path). When the model is deployed to a Model Serving endpoint, MLflow downloads these artifacts to the serving container's local filesystem. `context.artifacts["system_prompt"]` returns the LOCAL FILE PATH on the serving container where `system_prompt.txt` was copied — NOT the original relative path, NOT the file contents, and NOT a DataFrame. The code then uses this path as `self.system_prompt = context.artifacts["system_prompt"]` — but note this actually stores the file path, so to get the file contents the developer would need `open(context.artifacts["system_prompt"]).read()`. A is wrong because the original relative path `"./system_prompt.txt"` is the LOG-time path — it does not exist in the serving container; MLflow copies and relocates the file. C is wrong because `context.artifacts` provides paths, not file contents — the developer must explicitly read the file using the path. D is wrong because MLflow artifacts are files, not DataFrames; Databricks Model Serving does not convert file artifacts to DataFrames. **Source:** Section 4: Assembling and

Question 22

Difficulty: Advanced

A team uses OAuth M2M (Machine-to-Machine) authentication for their production application to call a Databricks Model Serving endpoint. Why is OAuth M2M preferred over using a personal access token (PAT) for this production scenario?

- **A)** OAuth M2M tokens are valid for 30 days without renewal, while PATs expire after 24 hours by default — the longer validity period reduces operational overhead of credential rotation in production systems.
- **B)** OAuth M2M uses a Service Principal identity, which has workspace-level permissions that PATs lack — Service Principals can access more Unity Catalog resources than user-level PATs are permitted to access.
- **C)** OAuth M2M generates short-lived tokens automatically via a client credentials flow, eliminating the need to store long-lived credentials — a Service Principal's `client_id` and `client_secret` grant access without persisting a long-lived token, reducing the blast radius of credential compromise.
- **D)** OAuth M2M is the only authentication method supported by Databricks Model Serving endpoints — PATs are only valid for the Databricks REST API for notebook and cluster management, not for Model Serving inference calls.

Correct Answer: C **Explanation:** C is correct. OAuth M2M (Machine-to-Machine) authentication is preferred for production because it uses a Service Principal with short-lived, auto-refreshed tokens generated via the OAuth 2.0 Client Credentials flow. The application stores only the Service Principal's `client_id` and `client_secret` (not a long-lived token), and the OAuth library automatically exchanges these for short-lived access tokens (typically valid for 1 hour) and refreshes them. If a token is compromised, it expires quickly — limiting the blast radius. PATs are long-lived tokens (configurable, often 90+ days) that are more dangerous if leaked. A is wrong because PAT expiry is configurable (not fixed at 24 hours), and OAuth M2M tokens are SHORT-lived (1 hour), not 30 days — the advantage is that short-lived tokens are MORE secure, not less convenient. B is wrong because PATs and Service Principals can be granted identical Unity Catalog permissions — the distinction is identity type and token management, not permission scope. D is wrong because PATs are also supported for Model Serving endpoint inference calls — OAuth M2M is preferred, not mandatory. **Source:** Section 4: Assembling and Deploying – Objective 2: Control access to resources from model serving endpoints — docs.databricks.com (search: "Model serving endpoint permissions Databricks")

Question 23

Difficulty: Advanced

A developer logs a LangChain chain with:

```
mlflow.langchain.log_model(  
    lc_model=chain,  
    artifact_path="rag_chain",  
    input_example={"context": "Sample context text.", "question": "What is the policy?"}  
)
```

Later, a colleague loads the model with `mlflow.pyfunc.load_model(model_uri)` and calls `model.predict(input_data)`. What format must `input_data` be in, and why?

- **A)** `input_data` must be a Python dictionary `{"context": "...", "question": "..."} —` `mlflow.pyfunc.load_model` always accepts raw dictionaries as inputs for LangChain models.
- **B)** `input_data` must be a Pandas DataFrame with columns matching the input schema (i.e., `pd.DataFrame([{"context": "...", "question": "..."}])`) — MLflow's pyfunc interface standardizes all model inputs as DataFrames, and the logged `input_example` defines the expected column names.
- **C)** `input_data` must be a JSON string `'{"context": "...", "question": "..."}'` that `mlflow.pyfunc.load_model` automatically parses before passing to the chain's `invoke` method.
- **D)** `input_data` can be any Python object — `mlflow.pyfunc.load_model` for LangChain models bypasses MLflow's type-checking system and passes whatever object is provided directly to the chain's `invoke()` method without any schema validation.

Correct Answer: B Explanation: B is correct. `mlflow.pyfunc.load_model()` returns an MLflow pyfunc model that implements the standard pyfunc interface. For this interface, `predict()` expects a Pandas DataFrame whose column names match the model signature's input schema. The `input_example` provided at log time (`{"context": "...", "question": "..."}`) is used to infer the model signature, which defines two string columns: `context` and `question`. The caller must provide a DataFrame with those exact column names: `pd.DataFrame([{"context": "actual context", "question": "actual question"}])`. A is wrong because the pyfunc interface does NOT accept raw Python dictionaries — it requires a Pandas DataFrame; passing a raw dict raises a signature validation error. C is wrong because raw JSON strings are not the expected input format for `pyfunc.predict()` — while JSON is the wire format for REST API calls to the serving endpoint, the local Python interface requires a DataFrame. D is wrong because pyfunc enforces the model signature when a signature is present (which it is, since `input_example` was provided) — it validates that the input DataFrame has the correct columns and types. **Source:** Section 4: Assembling and Deploying – Objective 3 & 4: Code a simple chain and RAG elements — docs.databricks.com (search: "Log and load LangChain models MLflow Databricks")

Question 24

Difficulty: Advanced

A Delta Sync Vector Search index uses `pipeline_type="CONTINUOUS"`. The source Delta table receives 10,000 new document chunks per hour via a streaming Spark job. A developer asks: "At what cost does CONTINUOUS sync come compared to TRIGGERED, and when does TRIGGERED become the right choice?"

- **A)** CONTINUOUS sync has zero additional cost compared to TRIGGERED sync — the Vector Search endpoint incurs the same fixed hourly compute charge regardless of whether CONTINUOUS or TRIGGERED mode is selected.
- **B)** CONTINUOUS sync incurs higher ongoing compute costs because the sync pipeline runs persistently, consuming Vector Search endpoint resources continuously to monitor and process Delta table changes. TRIGGERED sync is more cost-efficient when data freshness requirements allow periodic (e.g., daily or weekly) updates rather than near-real-time indexing.
- **C)** CONTINUOUS sync is cheaper than TRIGGERED sync because it processes changes incrementally (small batches), while TRIGGERED sync must reprocess the entire index from scratch on each sync, consuming more total compute per update cycle.
- **D)** TRIGGERED sync always costs exactly the same per-sync as CONTINUOUS mode costs per-hour — the cost difference depends only on how frequently the team triggers manual syncs, not on the pipeline type itself.

Correct Answer: B **Explanation:** B is correct. CONTINUOUS sync maintains an always-running sync pipeline that monitors the source Delta table's Change Data Feed in near real time and processes new changes as they arrive. This pipeline consumes Vector Search endpoint compute resources continuously — even during low-activity periods. TRIGGERED sync only consumes resources when explicitly invoked — the sync process runs to completion and then stops, consuming no resources until the next trigger. For workloads where data freshness requirements allow periodic updates (e.g., a knowledge base updated daily, a product catalog refreshed weekly), TRIGGERED sync is significantly more cost-efficient. CONTINUOUS mode is justified when freshness is critical (e.g., indexing news articles for a real-time news chatbot). A is wrong because CONTINUOUS and TRIGGERED sync do NOT have the same cost — CONTINUOUS incurs ongoing compute costs from the persistent pipeline. C is wrong because TRIGGERED sync is typically incremental too (using CDF to process only changed rows), not a full reindex; and TRIGGERED is cheaper for infrequent updates, not more expensive. D is wrong because the cost difference is inherent to the pipeline type (always-running vs. on-demand), not solely determined by trigger frequency. **Source:** Section 4: Assembling and Deploying – Objective 10: Configure vector search based on requirements — docs.databricks.com (search: "Mosaic AI Vector Search configuration")

Question 25

Difficulty: Advanced

A team's CI/CD pipeline uses Databricks Asset Bundles. The `databricks.yml` defines three environments: `dev`, `staging`, and `production`. The developer runs `databricks bundle deploy --target staging`. What happens, and what safety does this provide?

- **A)** `databricks bundle deploy --target staging` deploys ALL resources defined in `databricks.yml` to the `staging` workspace or the staging-specific resource prefix, using staging-specific variable values (e.g., endpoint names, catalog names) defined in the target configuration — it does NOT touch the `dev` or `production` environments.
- **B)** `databricks bundle deploy --target staging` validates the `databricks.yml` syntax only but does not deploy any resources — the `--target` flag is for specifying which environment's configuration to validate, and deployment requires an additional `--execute` flag.
- **C)** `databricks bundle deploy --target staging` deploys to staging AND automatically promotes validated resources to production if the staging deployment succeeds within a 5-minute health check window — this implements automatic blue-green promotion.
- **D)** `databricks bundle deploy --target staging` replaces the `dev` resources with staging resources — in Databricks Asset Bundles, each new deployment replaces the previous environment tier, migrating resources upward through the promotion pipeline.

Correct Answer: A **Explanation:** A is correct. Databricks Asset Bundles support multi-environment deployment through target configurations in `databricks.yml`. Each target can specify different variable values (e.g., `catalog: staging_catalog`, `model_name: rag_app_staging`) and different workspace URLs. When `databricks bundle deploy --target staging` is run: (1) DABs uses the `staging` target's variable values to parameterize the resource definitions. (2) It deploys only to the staging environment — `dev` and `production` are completely untouched. (3) This provides the safety guarantee that staging deployments are isolated — you can test staging without risk of breaking production. The promotion pipeline typically requires an explicit second `databricks bundle deploy --target production` command after staging validation. B is wrong because `databricks bundle validate` (without `deploy`) is the syntax-only validation command — `bundle deploy` always executes the deployment. C is wrong because DABs never automatically promotes between environments — each environment requires its own explicit `deploy` command; automatic promotion would be dangerous for production systems. D is wrong because environments in DABs are independent — deploying to staging does not modify `dev` resources; each target deploys to its own isolated configuration. **Source:** Section 4: Assembling and Deploying – Objective 12: Apply CI/CD best practices — docs.databricks.com (search: "Databricks Asset Bundles CI/CD")

Question 26

Difficulty: Advanced

A company has a knowledge base of 8 million document chunks. They need to choose Vector Search index configuration. Requirements: search latency must be under 50ms at P95, the knowledge base is updated once per week, and embedding costs must be minimized. What configuration satisfies all three requirements?

- **A)** Delta Sync Index, `pipeline_type="CONTINUOUS"`, Databricks-managed embeddings — continuous sync ensures near-real-time freshness, and Databricks-managed embeddings minimize per-query embedding computation time to achieve sub-50ms latency.
- **B)** Delta Sync Index, `pipeline_type="TRIGGERED"` (triggered weekly), self-managed embeddings pre-computed in batch and stored in the Delta table, with a correctly sized Vector Search endpoint for the 8M embedding index — TRIGGERED aligns with weekly update frequency, self-managed batch embedding minimizes cost, and endpoint sizing handles the latency requirement.
- **C)** Direct Vector Access Index with `pipeline_type="CONTINUOUS"` and self-managed real-time embeddings — the direct access API provides lower query latency than Delta Sync indexes, and continuous embedding updates ensure the index reflects the weekly knowledge base additions.
- **D)** Delta Sync Index, `pipeline_type="TRIGGERED"`, Databricks-managed embeddings, with a single Small-tier Vector Search endpoint — the managed embedding model ensures consistent vector quality, and a single Small endpoint is sufficient for 8 million embeddings at sub-50ms latency.

Correct Answer: B Explanation: B is correct. Matching requirements to configuration: (1) **< 50ms P95 latency** — achieved by correctly sizing the Vector Search endpoint (more nodes for a larger index) and using pre-computed self-managed embeddings (no embedding latency at query time — the query embedding is computed once and the ANN search runs against pre-indexed vectors). (2) **Updated once per week** — TRIGGERED mode is ideal; set up a weekly sync schedule. CONTINUOUS mode would waste compute resources running a persistent sync pipeline for a knowledge base that only changes weekly. (3) **Minimize embedding costs** — self-managed embeddings computed in batch (once per week on the new/changed chunks only) using cost-efficient batch processing is much cheaper than Databricks-managed embeddings which compute embeddings every time new data is written. A is wrong because CONTINUOUS sync wastes compute for a weekly-updated knowledge base, and Databricks-managed embeddings may not minimize cost for an 8M-chunk index. C is wrong because Direct Vector Access does not natively support CONTINUOUS sync — it requires manual embedding upload via API calls; and the combination described is not a valid architecture. D is wrong because a single Small-tier endpoint is likely insufficient for 8 million embeddings at sub-50ms P95 latency under concurrent query load — endpoint sizing must match the index size. **Source:** Section 4: Assembling and Deploying – Objective 10: Configure vector search based on requirements — docs.databricks.com (search: "Mosaic AI Vector Search configuration" and "Vector Search endpoint sizing")

Question 27

Difficulty: Advanced

A developer registers a model to Unity Catalog and then creates a Model Serving endpoint pointing to it. A week later, they need to update the underlying model (new version 2 registered). What is the zero-downtime upgrade process for the serving endpoint?

- **A)** Delete the existing serving endpoint, create a new endpoint pointing to version 2, and update the application's endpoint URL — the brief downtime during endpoint creation is unavoidable for model version updates.
- **B)** In the Model Serving UI or via REST API, add version 2 as a new served entity on the existing endpoint with a small traffic percentage (e.g., 10%), monitor its quality and latency metrics, then gradually shift traffic from version 1 to version 2 until version 2 receives 100% — then remove version 1 as a served entity.
- **C)** Re-register version 1 in Unity Catalog with the same version number but updated model artifacts — MLflow will detect the artifact change and automatically hot-reload the serving endpoint within 2 minutes without downtime.
- **D)** Create a Databricks Workflow that calls `mlflow.pyfunc.load_model()` on version 2 and assigns it to the endpoint's active model slot — the Workflow's model swap is atomic and provides zero-downtime switching.

Correct Answer: B **Explanation:** B is correct. Databricks Model Serving supports traffic splitting between multiple model versions on the same endpoint — this is the canonical zero-downtime upgrade pattern (canary/blue-green deployment). The process: (1) Register model version 2 to Unity Catalog. (2) Via the Serving UI or REST API, add version 2 as a new served entity with a small traffic split (e.g., 10% to v2, 90% to v1). (3) Monitor v2's metrics (latency, error rate, quality from Inference Tables). (4) Gradually shift traffic: 25/75, 50/50, 75/25, 100/0. (5) Once 100% is on v2, remove v1. Throughout this process, at least one version is always serving — zero downtime. A is wrong because creating a new endpoint with downtime is the worst-case approach — traffic splitting avoids this entirely. C is wrong because MLflow model versions in Unity Catalog are immutable — you cannot update artifacts under the same version number; each change creates a new version. D is wrong because `mlflow.pyfunc.load_model()` is for loading models into Python code, not for configuring which version an active serving endpoint uses; endpoint model management is done via the Serving API. **Source:** Section 4: Assembling and Deploying – Objective 7: Identify how to serve an LLM application — docs.databricks.com (search: "Databricks Foundation Model APIs" and "Provisioned throughput model serving")

Question 28

Difficulty: Advanced

A developer builds a Streamlit app hosted on Databricks Apps that calls a Databricks Model Serving endpoint. When a user logs in to the App via SSO, they want the app to call the endpoint using the user's identity (not a Service Principal) so Unity Catalog access controls are enforced per-user. What authentication mechanism does Databricks Apps use to enable this?

- **A)** Databricks Apps requires users to manually paste their personal access tokens (PAT) into the Streamlit app's sidebar input field, which the app stores in `st.session_state` and uses for all subsequent Model Serving API calls during the session.
- **B)** Databricks Apps automatically provides the logged-in user's short-lived OAuth token to the application code via the Databricks SDK's `WorkspaceClient()` context, which the app uses to call the Model Serving endpoint on behalf of the authenticated user.
- **C)** Databricks Apps creates a dedicated Service Principal for each user at login time, which makes all API calls on behalf of that user — the Service Principal is destroyed when the user session ends.
- **D)** Databricks Apps uses anonymous pass-through authentication — the Model Serving endpoint receives calls without any user identity, and Unity Catalog applies workspace-level (not user-level) access controls to all app-originated requests.

Correct Answer: B **Explanation:** B is correct. One of Databricks Apps' key architectural advantages is built-in SSO with identity pass-through. When a user authenticates to the Databricks App via SSO (Databricks identity), the App runtime automatically provides the authenticated user's OAuth token to the application code through the `WorkspaceClient()` context. The Streamlit app can then instantiate `WorkspaceClient()` without any manual credential configuration, and all API calls (to Model Serving endpoints, Unity Catalog, etc.) are made using the user's identity — meaning Unity Catalog row-level security, column masking, and model serving permissions are enforced per-user. A is wrong because requiring users to paste PATs is a security anti-pattern — Databricks Apps handles authentication automatically via SSO, eliminating the need for user-managed tokens. C is wrong because Databricks Apps does not create per-user Service Principals at login time — the user's own OAuth identity is passed through. D is wrong because Databricks Apps does not use anonymous authentication — a core value proposition is that user identity is preserved end-to-end for security and compliance. **Source:** Section 4: Assembling and Deploying – Objective 15: Develop an appropriate interactive user-facing interface — docs.databricks.com (search: "Databricks Apps deploy agent")

Question 29

Difficulty: Advanced

A team stores conversation history for a customer service agent in Lakebase using LangGraph's Checkpointer with `thread_id = user_session_id`. A user returns 3 days later and starts a new session. What happens to their previous conversation history, and how does the agent access it?

- **A)** The previous conversation history is gone — LangGraph's Checkpointer only persists history within the same `thread_id` session, and once the session ends, all state is evicted from memory and is unrecoverable.
- **B)** The previous conversation history is preserved in Lakebase (durable persistence). When the user starts a new session, the agent can either: (a) load the same `thread_id` to resume the exact previous conversation state, or (b) use a separate long-term memory Store to retrieve key facts from past sessions without loading the full history.
- **C)** The previous conversation history is automatically deleted by Lakebase's 24-hour data retention policy — Lakebase is an in-memory cache, not a durable database, and all data is lost after the configured retention period expires.
- **D)** The previous conversation history is preserved but requires a full table scan of the Lakebase conversation table because thread IDs are not indexed by default in Lakebase — performance degrades linearly with the number of past sessions stored.

Correct Answer: B **Explanation:** B is correct. Lakebase is a fully managed, durable Postgres-compatible database — data persists indefinitely until explicitly deleted. LangGraph's Checkpointer writes conversation state (all message turns) to Lakebase keyed by `thread_id`. Because Lakebase is durable: (1) When the user returns 3 days later, all their previous conversation turns are still in Lakebase. (2) The agent can load the same `thread_id` to perfectly resume the conversation as if no time had passed (short-term continuity). (3) Alternatively, for cross-session learning, the agent can use LangGraph's Store API to write and retrieve key facts (preferences, unresolved issues) across sessions using a persistent user-keyed namespace. A is wrong because Lakebase is a durable database, not an in-memory session store — data persists across session endings; persistence is the entire reason for using Lakebase instead of in-memory storage. C is wrong because Lakebase is NOT an in-memory cache — it is a serverless managed Postgres database with standard database durability guarantees; there is no 24-hour retention policy. D is wrong because Postgres (which Lakebase is compatible with) supports indexed lookups — thread IDs can be indexed for $O(\log n)$ retrieval, not requiring a full table scan. **Source:** Section 4: Assembling and Deploying – Objective 11: Configure a persistent datastore for memory — docs.databricks.com (search: "Agent memory Databricks" and "Lakebase Databricks Postgres")

Question 30

Difficulty: Advanced

A developer writes a CI/CD test for their RAG chain component. They want to test the retriever independently (without calling the LLM) to verify that given a specific query, the correct documents are returned from Vector Search. What is the correct testing approach?

- **A)** The retriever cannot be tested independently — because Vector Search is embedded within the LangChain chain, the only way to test it is to run the full end-to-end chain and check if the LLM's answer reflects the correct documents.
- **B)** Instantiate the `DatabricksVectorSearch` retriever directly in the test, call `retriever.invoke("test query")`, and assert that the returned documents include the expected content — this tests the retriever in isolation without invoking the LLM or prompt template.
- **C)** Use `mlflow.evaluate()` to test the retriever in isolation — pass the retriever as the `model` argument and provide the test query as the evaluation dataset; MLflow will run the retriever and return retrieval metrics.
- **D)** Create a mock `DatabricksVectorSearch` retriever using Python's `unittest.mock` library and assert that `retriever.get_relevant_documents()` is called with the correct query — this validates the chain's retriever invocation without connecting to a real Vector Search index.

Correct Answer: B Explanation: B is correct. LangChain's LCEL architecture makes components independently invokable. The `DatabricksVectorSearch` retriever implements LangChain's `BaseRetriever` interface with an `invoke()` method. In a unit or integration test, you can instantiate the retriever directly (pointing at the staging Vector Search index), call `retriever.invoke("What is the refund policy?")`, and assert properties of the returned document list — checking that the expected chunks are present, that the correct number of results was returned, and that metadata is correct. This is a true integration test of the retriever component without LLM involvement. A is wrong because the LCEL `|` operator creates composable, independently invokable components — each can be tested in isolation; testing only the full chain provides poor isolation when debugging retrieval failures. C is wrong because `mlflow.evaluate()` is for evaluating a complete model/chain against a quality benchmark dataset — it is not designed for unit testing a single retriever component in isolation. D is wrong because mocking the retriever only verifies that the chain CALLS the retriever, not that the retriever returns CORRECT results from a real Vector Search index — mocks are for unit tests of chain composition logic, not for verifying retrieval quality. **Source:** Section 4: Assembling and Deploying – Objective 12: Apply CI/CD best practices — docs.databricks.com (search: "Databricks Asset Bundles CI/CD")

Question 31

Difficulty: Advanced

A developer builds a Custom MCP Server using Databricks Apps and FastAPI. The server exposes two tools: `search_internal_docs()` and `submit_support_ticket()`. A Supervisor agent connects to it via the MCP protocol. What is a key advantage of hosting the Custom MCP Server on Databricks Apps versus a standalone external server?

- **A)** Databricks Apps Custom MCP Servers automatically register their tools in Unity Catalog, making them discoverable by any agent in the workspace without requiring manual tool registration in the Supervisor's tool list.
- **B)** Hosting on Databricks Apps keeps the Custom MCP Server within the Databricks security perimeter — it benefits from built-in SSO, Databricks identity, Unity Catalog access controls, and Unity AI Gateway governance for all tool invocations.
- **C)** Databricks Apps Custom MCP Servers bypass the Unity AI Gateway and connect directly to the Supervisor agent's LangGraph runtime, eliminating the routing overhead that External MCP Servers incur through the Gateway.
- **D)** Custom MCP Servers on Databricks Apps are co-located on the same compute cluster as the Model Serving endpoint, reducing network latency from the Supervisor to the MCP server to near-zero compared to external hosting.

Correct Answer: B **Explanation:** B is correct. Hosting a Custom MCP Server on Databricks Apps provides all the governance and security benefits of the Databricks platform: (1) **SSO and Databricks identity** — users accessing tools via the MCP server are authenticated via Databricks identity, maintaining end-to-end audit trails. (2) **Unity Catalog access controls** — the MCP server can call Unity Catalog resources (Delta tables, ML models, Vector Search) using properly governed permissions. (3) **Unity AI Gateway** — all MCP traffic routes through the Gateway for rate limiting, cost monitoring, and compliance policies. (4) **No separate infrastructure** — no external server to deploy, scale, or secure separately. A is wrong because Custom MCP Server tools on Databricks Apps are NOT automatically registered in Unity Catalog — the Supervisor agent's tool list must be explicitly configured to point to the MCP server URL. C is wrong because Custom MCP Servers on Databricks Apps DO route through Unity AI Gateway — all MCP server traffic is governed through the Gateway, which is a feature (governance), not a limitation. D is wrong because Databricks Apps are not co-located on the same compute as Model Serving endpoints — they run on separate managed infrastructure; the latency benefit is not "near-zero" due to co-location. **Source:** Section 4: Assembling and Deploying – Objective 13: Integrate managed, external, and custom MCP servers — docs.databricks.com (search: "MCP servers Databricks" and "Unity Catalog MCP services")

Question 32

Difficulty: Advanced

A developer needs to add a Unity Catalog Function as a tool to a LangGraph agent. The function `main.agents.calculate_tax(amount DOUBLE, region STRING) RETURNS DOUBLE` is already registered. What is the correct Databricks SDK approach to make this function available as an LLM tool?

- **A)** Import the function using `from databricks.sdk.service.catalog import FunctionAPI` and call `FunctionAPI.to_langchain_tool(function_name="main.agents.calculate_tax")` to get a LangChain-compatible tool object.
- **B)** Use `UCFunctionToolkit` from `unitycatalog.ai.langchain` or the Databricks SDK's UC function toolkit — it automatically fetches the function's signature from Unity Catalog, creates a type-safe tool wrapper, and registers it with the LangGraph agent's tool list.
- **C)** Convert the UC Function to a LangChain tool by manually writing a Python wrapper function with a matching signature, decorating it with `@tool`, and documenting its input/output schema in the docstring — Unity Catalog functions cannot be directly wrapped automatically.
- **D)** Call the UC Function directly from inside a LangGraph node by executing `spark.sql("SELECT main.agents.calculate_tax(1000, 'CA')")` and return the Spark DataFrame result to the LLM as a JSON-serialized string.

Correct Answer: B **Explanation:** B is correct. Databricks provides `UCFunctionToolkit` (available in the `unitycatalog-ai` package and Databricks SDK extensions) specifically for converting Unity Catalog Functions into LangChain-compatible tools. When initialized with a function name (e.g., `main.agents.calculate_tax`), `UCFunctionToolkit` automatically: (1) Fetches the function's metadata from Unity Catalog (parameter names, types, return type, description). (2) Creates a type-safe LangChain `Tool` object with the correct input schema. (3) Handles execution by calling the UC Function via the Databricks REST API when the LLM generates a tool call. The resulting tool can be added directly to a LangGraph agent's tool list. A is wrong because `FunctionAPI.to_langchain_tool()` is not a real method in the Databricks SDK — `UCFunctionToolkit` is the correct abstraction. C is wrong because manual wrapping with `@tool` is the pre-toolkit workaround — `UCFunctionToolkit` automates this completely, including type inference from the UC Function's registered schema. D is wrong because executing Spark SQL inside a LangGraph node and returning a raw DataFrame string is a fragile anti-pattern — it doesn't create a proper tool that the LLM's tool-calling interface can invoke through structured JSON. **Source:** Section 4: Assembling and Deploying – Objective 13: Integrate managed, external, and custom MCP servers — docs.databricks.com (search: "MCP servers Databricks")

Question 33

Difficulty: Advanced

A team wants to implement unit testing for their prompt template as part of their CI/CD pipeline, before deploying it to production. The prompt is registered in the MLflow Prompt Registry under the `staging` alias. How should the CI/CD test validate the prompt behavior before promoting to production?

- **A)** Use `mlflow.load_prompt("prompts:/rag_prompt/staging")` in a pytest test to load the staging prompt, format it with test input variables, and use an LLM-as-a-judge or assertion-based check to verify the formatted prompt meets expected format and instruction quality.
- **B)** The prompt cannot be unit tested independently — because prompts are natural language strings, behavioral validation requires a full MLflow evaluation run with the complete chain, not an isolated unit test.
- **C)** Use `mlflow.compare_prompts(alias1="staging", alias2="production")` to run a built-in A/B comparison that automatically evaluates both prompt versions against a standard benchmark and returns a winner recommendation.
- **D)** Unit test the prompt by checking that `mlflow.load_prompt("prompts:/rag_prompt/staging").text` is not equal to `mlflow.load_prompt("prompts:/rag_prompt/production").text` — confirming that the staging prompt is different from the current production version before promoting.

Correct Answer: A **Explanation:** A is correct. The correct CI/CD approach for prompt validation is: (1) `mlflow.load_prompt("prompts:/rag_prompt/staging")` loads the staging prompt version. (2) Use the prompt's `.format()` or `.template` attribute to assemble test prompts with known test input variables. (3) Apply assertions to verify the formatted prompt: check that required variables are correctly substituted, that system instructions are present, that the format matches expected structure. (4) For higher-confidence validation, run the formatted prompt against an LLM on a small test set and use LLM-as-a-judge or regex-based checks to validate the output quality. This is a lightweight, fast test that can run in CI without a full `mlflow.evaluate()` cycle. B is wrong because prompts CAN be tested independently — at minimum, template formatting (variable substitution, structure) can be validated without any LLM call; and component-level testing is a CI/CD best practice. C is wrong because `mlflow.compare_prompts()` is not a real MLflow API function — prompt comparison in MLflow is done by loading and examining prompts in code, not via a built-in comparison API. D is wrong because checking that the prompts are merely DIFFERENT is a trivially weak test — it doesn't validate that the staging prompt is BETTER or even CORRECT; it would pass even if the staging prompt was accidentally emptied. **Source:** Section 4: Assembling and Deploying – Objective 12 & 14: CI/CD best practices and prompt lifecycle — docs.databricks.com (search: "MLflow Prompt Registry aliases" and "Databricks Asset Bundles CI/CD")

Question 34

Difficulty: Advanced

A developer reviews the following `ai_query()` usage in a Databricks SQL notebook:

```
SELECT
  id,
  ai_query(
    'databricks-meta-llama-3-70b-instruct',
    CONCAT('Extract the contract type from this text. Return only: NDA, SLA, MSA, or OTHER. Text
  ) AS contract_type
FROM main.legal.contracts
WHERE processed_date IS NULL;
```

What optimization issues, if any, exist with this query for a table with 500,000 unprocessed contracts?

- **A)** No optimization issues — `ai_query()` automatically batches all 500,000 rows into optimal request sizes, parallelizes across serverless compute, and handles rate limiting internally; the query will complete efficiently.
- **B)** The `CONCAT` function is a performance bottleneck — replacing it with a prepared statement parameter binding reduces SQL compilation overhead and improves throughput for large-scale `ai_query()` runs.
- **C)** The query is missing a `LIMIT` clause — `ai_query()` on serverless SQL Warehouses requires a `LIMIT` to prevent runaway costs; without a `LIMIT`, the query will fail with a budget exceeded error after processing 10,000 rows.
- **D)** The prompt includes the full contract text inline which could exceed the model's context window for very long contracts — the developer should add a `LENGTH(contract_text) < 10000` filter or apply pre-processing to truncate or summarize long contracts before `ai_query()` classification.

Correct Answer: D **Explanation:** D is correct. The most significant issue is that `contract_text` is passed directly without any length guard. Legal contracts can be thousands or tens of thousands of words long. If any contract's text (plus the prompt prefix) exceeds the LLM's context window, the `ai_query()` call for that row will fail — causing the entire query to fail or producing NULL results for those rows. Best practices include: adding a `LENGTH(contract_text) < <token_limit_estimate>` filter, or using a pre-processing step to extract the first N characters of the contract header for classification (contracts often have their type identified early). A is wrong because while `ai_query()` does parallelize and handle rate limiting, it does NOT handle context window overflows — rows that exceed the context window will generate errors, not gracefully truncate. B is wrong because `CONCAT` is a standard SQL string function with negligible overhead compared to model inference time — it is not a performance bottleneck in this context. C is wrong because `ai_query()` does not require a `LIMIT` clause and will not fail with a budget error after 10,000 rows — it processes all qualifying rows; budget controls are managed through Unity AI Gateway cost budgets, not query-level `LIMIT` constraints. **Source:** Section 4: Assembling and Deploying – Objective 9: Identify batch inference workloads and apply `ai_query()` — docs.databricks.com (search: "ai_query function Databricks SQL" and "Batch inference AI functions")

Question 35

Difficulty: Proficiency

A principal engineer designs a complete deployment architecture for a RAG application on Databricks. They must satisfy: (1) vector search updates must reflect new documents within 5 minutes of them being added to the source Delta table, (2) the serving endpoint must never store credentials in plain text, (3) the application must be accessible to employees via their existing company SSO without separate login, (4) infrastructure must be version-controlled and reproducible across dev/staging/prod. Map each requirement to the correct Databricks feature.

- **A)** (1) VS CONTINUOUS sync → (2) Databricks Secrets as environment variables → (3) Databricks Apps with built-in SSO → (4) Databricks Asset Bundles in Git.
- **B)** (1) VS TRIGGERED sync triggered every 5 minutes → (2) Unity Catalog column encryption → (3) External web app with OAuth M2M → (4) Databricks Workflows deployment notebook.
- **C)** (1) VS CONTINUOUS sync → (2) Unity Catalog row-level security → (3) Genie app for Teams → (4) MLflow Projects.
- **D)** (1) VS TRIGGERED sync → (2) Databricks Secrets in `load_context()` → (3) Databricks Apps → (4) Delta Live Tables pipeline YAML.

Correct Answer: A **Explanation:** A is correct. Each requirement maps precisely: (1) **< 5 minutes freshness** → `pipeline_type="CONTINUOUS"` on the Vector Search Delta Sync index monitors the source Delta table's CDF in near real-time (typical lag < 1–2 minutes) — well within the 5-minute requirement. TRIGGERED sync requires manual triggers and cannot guarantee sub-5-minute freshness. (2) **No plain-text credentials** → Databricks Secrets Scope stores credentials encrypted; referencing them as environment variables in the endpoint configuration means the values are never visible in code, logs, or notebook cells. (3) **Company SSO without separate login** → Databricks Apps integrates with Databricks workspace SSO — employees log in with their existing company identity (via SAML/OIDC), with no separate password or registration. (4) **Version-controlled reproducible infrastructure** → Databricks Asset Bundles (DABs) define all Databricks resources in `databricks.yml` committed to Git, enabling `databricks bundle deploy` to reproduce the exact same infrastructure across dev/staging/prod. B is wrong because TRIGGERED every 5 minutes cannot guarantee sub-5-minute freshness (sync takes time after triggering), Unity Catalog column encryption is for data governance not credential management, and Workflows notebooks are not IaC. C and D have partial matches but contain incorrect mappings for multiple requirements. **Source:** Section 4: Assembling and Deploying – Objectives 2, 6, 10, 12, 15 — docs.databricks.com (search: "Databricks Asset Bundles CI/CD" and "Mosaic AI Vector Search configuration" and "Databricks Secrets model serving")

Question 36

Difficulty: Proficiency

A developer receives this error when registering an MLflow model to Unity Catalog:

```
MLflowException: Model 'main.ml_models.rag_chain' requires a model signature.  
Provide an input_example or explicitly define a signature using mlflow.models.infer_signature().
```

The model was logged with:

```
mlflow.langchain.log_model(lc_model=chain, artifact_path="rag_chain")
```

What is the MINIMUM fix, and what two alternative approaches exist?

- **A)** The minimum fix is to add `registered_model_name="main.ml_models.rag_chain"` to the `log_model()` call — the registration error occurs because the model name was not specified at log time, causing MLflow to attempt registration without namespace context.
- **B)** Minimum fix: Add `input_example={"question": "What is the return policy?", "context": "Our return policy..."}` to `log_model()` — MLflow automatically infers the signature from this example. Alternative 1: Manually create a signature with `infer_signature(inputs, outputs)` and pass it to `log_model(signature=...)`. Alternative 2: Call `mlflow.models.add_signature(model_uri, signature)` to add a signature to an already-logged model without re-logging.
- **C)** Minimum fix: Run `mlflow.set_registry_uri("databricks-uc")` before the `log_model()` call — the error occurs because Unity Catalog's registry is not selected and the model was sent to the legacy registry which rejects signatures.
- **D)** The error cannot be fixed without deleting the current model version and re-running the full training and evaluation pipeline — model signatures cannot be added retrospectively to logged MLflow models in Unity Catalog.

Correct Answer: B **Explanation:** B is correct. Three approaches to fix the signature error: (1) **Minimum fix** — Add `input_example={...}` to `mlflow.langchain.log_model()`. MLflow 2.5+ automatically infers the model signature from the provided example (inspecting its data types and column names) and attaches it to the model artifact. This is the recommended approach for its simplicity. (2) **Alternative 1** — Explicitly define the signature by running the chain on sample data, capturing inputs and outputs, and using `mlflow.models.infer_signature(model_input=sample_input, model_output=sample_output)`, then pass it as `log_model(signature=signature)`. (3) **Alternative 2** — Use `mlflow.models.add_signature(model_uri="runs:/<run_id>/rag_chain", signature=signature)` to attach a signature to an ALREADY-LOGGED model without re-running `log_model()`. A is wrong because `registered_model_name` specifies where to register the model, not what signature it has — adding this parameter does not fix the missing signature. C is wrong because `mlflow.set_registry_uri("databricks-uc")` is needed to point to Unity Catalog, but the error is specifically about the missing signature, not registry selection. D is wrong because signatures CAN be added to already-logged models using `mlflow.models.add_signature()` — no re-logging is required. **Source:** Section 4: Assembling and Deploying – Objective 5: Register the model to Unity Catalog using MLflow — docs.databricks.com (search: "Register model Unity Catalog MLflow" and "MLflow model signature input example")

Question 37

Difficulty: Proficiency

An enterprise runs a RAG-based legal research assistant used by 500 attorneys. Requirements: (1) each attorney's search results must only include documents they are individually authorized to see (row-level security on the knowledge base), (2) the agent must remember each attorney's practice area from the last session, (3) new case law documents must be indexed within 2 hours of being added, (4) the application must run within the corporate IT security perimeter without external SaaS dependencies. Design the architecture.

- **A)** Vector Search Direct Access Index (for row-level security at query time) + Lakebase (cross-session attorney memory) + TRIGGERED sync triggered every 90 minutes (meets 2-hour freshness) + Databricks Apps (within the Databricks security perimeter).
- **B)** Vector Search Delta Sync Index with CONTINUOUS sync + Unity Catalog row-level security on the source Delta table (access filters propagate to Vector Search queries) + Lakebase Store for cross-session memory + Databricks Apps for the front-end within the Databricks security perimeter.
- **C)** External Elasticsearch cluster for row-level security + Redis for session memory + CONTINUOUS sync from Delta to Elasticsearch + Databricks Model Serving for the LLM component — this provides enterprise-grade security without relying on Databricks-native components.
- **D)** Vector Search Delta Sync Index with CONTINUOUS sync + no memory (each attorney re-enters their practice area per session, as Databricks does not support cross-session agent memory within the security perimeter) + Databricks Apps front-end.

Correct Answer: B **Explanation:** B is correct. Mapping requirements: (1) **Row-level security per attorney** → Unity Catalog row-level security policies on the source Delta table are propagated to Vector Search queries via the `filters` parameter in `similarity_search()`. The searching identity (attorney's UC identity) determines which rows they can retrieve — documents outside their authorization are excluded at the Vector Search layer. (2) **Cross-session memory** → LangGraph Store API writing attorney practice area preferences to Lakebase, keyed by attorney user ID. (3) **< 2-hour freshness** → CONTINUOUS sync ensures documents are indexed within minutes of being added — well within 2 hours. (4) **Within security perimeter** → Databricks Apps (built-in SSO, UC identity, within Databricks perimeter). A is wrong because a Direct Vector Access Index doesn't natively propagate Unity Catalog row-level security — the security controls are at the Unity Catalog source table, not at the index query level; also TRIGGERED at 90 minutes is a compromise when CONTINUOUS is more reliable. C is wrong because using Elasticsearch and Redis introduces external SaaS dependencies, violating requirement 4 (within corporate IT perimeter). D is wrong because Databricks does support cross-session memory via Lakebase — omitting memory is not a technical limitation but an architectural choice that directly violates requirement 2. **Source:** Section 4: Assembling and Deploying – Objectives 6, 8, 10, 11, 15 — docs.databricks.com (search: "Mosaic AI Vector Search configuration" and "Agent memory Databricks" and "Databricks Apps deploy agent")

Question 38

Difficulty: Proficiency

A data engineer implements batch inference using `ai_query()` on 2 million product descriptions to extract structured JSON attributes. The query works on a 1,000-row sample but fails on the full 2 million rows with `ModelNotAvailableException: Rate limit exceeded` errors. What is the correct remediation?

- **A)** Switch to a Provisioned Throughput endpoint with enough token-per-second capacity to handle the 2 million rows within the SQL query's timeout window — Provisioned Throughput eliminates rate limiting by dedicating capacity to your workload.
- **B)** Add a `WHERE MOD(id, 4) = 0` clause to the SQL query to process only 25% of the rows in the current run, then run the query 4 times with different modulus values to process all rows while staying within rate limits.
- **C)** Split the query into 2,000 separate SQL queries of 1,000 rows each using a Databricks Workflow with 2,000 Tasks, which distributes the load across 2,000 separate rate limit windows and avoids the per-minute token limit.
- **D)** Increase the serverless SQL Warehouse's concurrency setting from 1 to 10, which allows 10 parallel `ai_query()` executions per row and reduces the wall-clock time by 10×, staying within the model endpoint's rate limit per request.

Correct Answer: A **Explanation:** A is correct. The `Rate limit exceeded` error when scaling from 1K to 2M rows indicates that the pay-per-token endpoint's rate limit (requests per minute or tokens per minute) is being hit. The correct solution for production-scale batch inference is to use a Provisioned Throughput endpoint with capacity sized for the workload. Provisioned Throughput reserves a dedicated tokens-per-second (TPS) allocation — there is no throttling for traffic within the reserved capacity. The team should: (1) estimate total tokens needed (2M rows × avg tokens per row), (2) calculate required TPS to complete within the batch window, (3) provision the endpoint with that TPS. B is wrong because processing in 4 separate runs via modulus is a workaround that doesn't fix the underlying rate limit — each run still hits the same rate limit. C is wrong because 2,000 separate Workflow Tasks each running 1K-row queries doesn't avoid the rate limit — each task still calls the same endpoint, and 2,000 concurrent tasks would hit the rate limit faster, not slower. D is wrong because the warehouse concurrency setting controls how many SQL queries run simultaneously on the warehouse — it doesn't divide the rate limit; increasing concurrency actually INCREASES the rate of API calls, worsening the rate limit problem. **Source:** Section 4: Assembling and Deploying – Objective 9: Identify batch inference workloads and apply `ai_query()` — docs.databricks.com (search: "ai_query function Databricks SQL" and "Batch inference AI functions")

Question 39

Difficulty: Proficiency

A developer builds a sophisticated pyfunc model with pre-processing (input schema validation, PII sanitization) and post-processing (JSON parsing, confidence threshold filtering). During load testing, they discover that 12% of requests return incorrect results when inputs contain Unicode special characters (emoji, non-ASCII). Where in the pyfunc implementation should the Unicode normalization fix be applied?

- **A)** In `load_context()` — because Unicode normalization is a one-time configuration operation that sets the encoding context for all subsequent `predict()` calls, and it should be performed during the model's initialization phase.
- **B)** In the `_preprocess()` method called from `predict()` — because normalization must be applied to each individual inference request's input before the model call, and the `_preprocess()` step is exactly where per-request input transformations belong.
- **C)** In the MLflow model signature definition using `ColSpec(type=DataType.binary)` — declaring the input column as binary type forces MLflow to accept raw bytes and skip Unicode decoding, preventing the encoding issue before it reaches `predict()`.
- **D)** In the Model Serving endpoint's environment variable configuration using `PYTHONIOENCODING=utf-8` — setting the encoding environment variable makes all Python string operations in the serving container default to UTF-8, automatically fixing Unicode handling without code changes.

Correct Answer: B **Explanation:** B is correct. The `_preprocess()` method (or equivalently, the first lines of `predict()`) is the correct location for per-request input transformations — including Unicode normalization. Each request's input passes through `_preprocess()` before reaching the model, so normalization applied here ensures all model calls receive clean, normalized text. The typical fix: `import unicodedata; text = unicodedata.normalize('NFC', text)` applied to each input string. A is wrong because `load_context()` runs once at startup and does not have access to individual request data — you can load a Unicode normalizer object there, but you cannot normalize the per-request inputs in `load_context()`. C is wrong because changing the input signature to binary type forces callers to pre-encode strings as bytes — this is an API-breaking change that shifts the burden to callers and does not actually fix the normalization issue within the model. D is wrong because `PYTHONIOENCODING=utf-8` affects stdin/stdout encoding for the Python process — it does not automatically normalize Unicode characters in string variables; normalization (NFC, NFKC) is distinct from encoding and must be explicitly applied in code. **Source:** Section 4: Assembling and Deploying – Objective 1: Code a chain using a pyfunc model with pre- and post-processing — docs.databricks.com (search: "mlflow.pyfunc PythonModel custom" and "Deploy custom Python model Databricks Model Serving")

Question 40

Difficulty: Proficiency

A team's production RAG agent has been running for 6 months. A new regulation requires that all documents tagged with `classification = "confidential"` must never be retrievable by any user through the Vector Search index. The index currently contains 500,000 documents, of which 50,000 are newly classified as confidential. What is the correct remediation?

- **A)** Delete the confidential documents from the source Delta table, then trigger a full re-index of the Vector Search index — the full re-index rebuilds the index from scratch, ensuring the confidential document vectors are permanently removed.
- **B)** Update the source Delta table to remove the 50,000 confidential rows (using `DELETE FROM ... WHERE classification = "confidential"`), apply the `VACUUM` command to remove the underlying Parquet files, trigger a Vector Search sync — the sync processes the CDF delete operations and removes the corresponding vectors from the index.
- **C)** Add a `filter = "classification != 'confidential'"` metadata filter to all `similarity_search()` calls in the retriever code — this filter prevents confidential documents from being returned by queries even though their vectors remain indexed.
- **D)** Assign a Unity Catalog row-level security policy that blocks the `databricks-vector-search` service principal from reading confidential rows — this prevents the Vector Search service from indexing or returning the confidential document vectors.

Correct Answer: B **Explanation:** B is correct — and order matters. The correct sequence is: (1) `DELETE FROM main.schema.chunked_docs WHERE classification = 'confidential'` — removes the rows from the source Delta table (creates DELETE entries in the CDF). (2) `VACUUM main.schema.chunked_docs` (after removing the 7-day default retention) — physically deletes the underlying Parquet files so the confidential text is not in storage. (3) Trigger a Vector Search sync (`index.sync()` or automatic via CONTINUOUS mode) — the sync processes the CDF DELETE operations and removes the corresponding vector entries from the index. This fully eliminates the confidential vectors. A is wrong because a "full re-index from scratch" is expensive and slower than incremental CDF-based sync for 50,000 deleted rows — and more importantly, `VACUUM` is still needed to remove the underlying Parquet files. C is wrong because filtering at query time leaves the confidential vectors IN the index — a compromised or incorrectly implemented filter could expose them; true remediation requires removing the vectors from the index, not just filtering results. D is wrong because row-level security on the `databricks-vector-search` service principal would prevent future sync of those rows, but doesn't remove the vectors already indexed — the 50,000 existing vectors remain searchable. **Source:** Section 4: Assembling and Deploying – Objective 6 & 8: Create/query Vector Search index and key concepts — docs.databricks.com (search: "Create Mosaic AI Vector Search index" and "Mosaic AI Vector Search overview")

Question 41

Difficulty: Beginner

A developer has a source Delta table `main.docs.chunks` and wants to create a Vector Search index that automatically stays up-to-date as new chunks are appended. What prerequisite must be enabled on the Delta table before creating a Delta Sync index?

- **A)** The Delta table must have `delta.enableAutoOptimize = true` set as a table property — Auto Optimize compacts small files, which is required for Vector Search's incremental change detection.
- **B)** The Delta table must have Change Data Feed (CDF) enabled: `ALTER TABLE main.docs.chunks SET TBLPROPERTIES (delta.enableChangeDataFeed = true)` — CDF produces a changelog of inserts, updates, and deletes that the Delta Sync pipeline reads to keep the index current.
- **C)** The Delta table must be converted to a Unity Catalog Managed Table with `CONVERT TO MANAGED TABLE main.docs.chunks` — Vector Search Delta Sync indexes only work with Unity Catalog managed tables, not external tables.
- **D)** The Delta table must have row-level security disabled on the `chunk_text` column using `ALTER TABLE main.docs.chunks DROP ROW FILTER` — row filters block the Vector Search service from reading the full table content for indexing.

Correct Answer: B **Explanation:** B is correct. Change Data Feed (CDF) is a mandatory prerequisite for creating a Delta Sync Vector Search index. CDF records every data change (INSERT, UPDATE, DELETE) in a special `_change_data` directory alongside the Delta table's main data files. The Vector Search Delta Sync pipeline reads this CDF to identify which chunks are new, updated, or deleted, and processes only those changes — making incremental sync efficient for large tables. Without CDF enabled, the Vector Search service has no way to detect changes and the index creation fails. A is wrong because `enableAutoOptimize` compacts small files for query performance, but it is not required for Vector Search — CDF is the mandatory prerequisite. C is wrong because Delta Sync indexes work with both UC Managed and UC External tables — the requirement is CDF enablement, not table type conversion. D is wrong because row-level security is applied at query time (to restrict what data users can retrieve), not at index build time — it does not block the Vector Search service from indexing content for users authorized to access it. **Source:** Section 4: Assembling and Deploying – Objective 6: Create and query a Vector Search index — docs.databricks.com (search: "Create Mosaic AI Vector Search index")

Question 42

Difficulty: Beginner

A developer writes `mlflow.set_registry_uri("databricks-uc")` at the start of their MLflow logging script. What does this line accomplish?

- **A)** It sets the MLflow tracking server's backend store to Unity Catalog, routing all experiment runs, metrics, and artifacts to the Unity Catalog-managed MLflow tracking server rather than the workspace-local tracking server.
- **B)** It tells MLflow to register all subsequently logged models to the Unity Catalog Model Registry (using the three-level namespace) instead of the legacy workspace-level MLflow Model Registry — enabling Unity Catalog governance, versioning, and permissions on registered models.
- **C)** It enables Unity Catalog authentication for the current MLflow session, requiring all model logging operations to use the caller's Unity Catalog identity and pass through Unity Catalog access control checks.
- **D)** It migrates all existing models in the legacy MLflow Model Registry to Unity Catalog, performing a one-time bulk migration of model versions and their associated metadata to the Unity Catalog registry.

Correct Answer: B **Explanation:** B is correct. `mlflow.set_registry_uri("databricks-uc")` switches the target model registry from the legacy workspace-level MLflow Model Registry to the Unity Catalog Model Registry. After this call, any `mlflow.register_model()` call or `registered_model_name` parameter in `log_model()` will register the model as a Unity Catalog three-level namespace object (e.g., `catalog.schema.model_name`). The model then inherits Unity Catalog governance: RBAC permissions, cross-workspace discovery, lineage tracking, and model version management. A is wrong because `set_registry_uri` changes the MODEL REGISTRY destination, not the experiment/tracking server — the tracking server stores runs, metrics, and artifacts separately from the model registry. C is wrong because Unity Catalog authentication for the current session is established at workspace login, not by `set_registry_uri` — this call only affects where models are registered. D is wrong because `set_registry_uri` is not a migration command — it does not touch existing model registry entries; it only affects where FUTURE `register_model()` calls go. **Source:** Section 4: Assembling and Deploying – Objective 5: Register the model to Unity Catalog using MLflow — docs.databricks.com (search: "Register model Unity Catalog MLflow")

Question 43

Difficulty: Beginner

A developer asks: "What is the difference between a Vector Search Endpoint and a Vector Search Index?" Select the correct explanation.

- **A)** A Vector Search Endpoint is the query interface (the REST API URL) that client applications call to run similarity searches; a Vector Search Index is a physical storage object (similar to a database table) that the Endpoint queries internally.
- **B)** A Vector Search Endpoint is the compute cluster managed by Databricks that serves one or more indexes; a Vector Search Index is the actual data structure containing the embedded vectors, stored as a Unity Catalog object and served by the endpoint.
- **C)** A Vector Search Endpoint is a Databricks Model Serving endpoint configured to use the `embeddings` task type; a Vector Search Index is a Delta table with an added vector column, which the endpoint queries using Spark DataFrame operations.
- **D)** A Vector Search Endpoint and a Vector Search Index are interchangeable terms — both refer to the same resource that stores embeddings and exposes a similarity search API; the different names reflect different API versions.

Correct Answer: B **Explanation:** B is correct. These are two distinct resources with a parent-child relationship: (1) **Vector Search Endpoint** — a compute cluster managed by Databricks that provides the serving infrastructure. It must be created **FIRST** before any indexes can be created. One endpoint can serve multiple indexes. It is analogous to a database server. (2) **Vector Search Index** — the actual Unity Catalog object that stores the embedded vectors (as an ANN index structure), linked to a source Delta table via CDF, and served by the parent endpoint. It is analogous to a database table. The typical setup: one Vector Search Endpoint hosts multiple Vector Search Indexes for different knowledge bases. A is wrong because the Endpoint is not just a "REST API URL" — it is a compute cluster that runs the ANN search algorithm. The index is not just queried through the endpoint; the endpoint **IS** the compute that runs the search. C is wrong because Vector Search Endpoints are completely separate from Databricks Model Serving endpoints — they are a different service; and Vector Search Indexes are **NOT** Delta tables with vector columns, they are specialized ANN index data structures. D is wrong because Endpoint and Index are distinct resources with different APIs, different creation parameters, and different management lifecycles. **Source:** Section 4: Assembling and Deploying – Objective 8: Explain key concepts and components of Mosaic AI Vector Search — docs.databricks.com (search: "Mosaic AI Vector Search overview")

Question 44

Difficulty: Beginner

A developer wants to test their pyfunc model locally before deploying to a Model Serving endpoint. After logging the model with `mlflow.pyfunc.log_model()`, they load it with `mlflow.pyfunc.load_model(model_uri)`. What object does `load_model()` return, and how do they call it?

- **A)** It returns an instance of the developer's `CustomRAGChain` class directly — they can call any method defined on the class, including private helper methods like `_preprocess()` and `_postprocess()`.
- **B)** It returns an `mlflow.pyfunc.PyFuncModel` wrapper object — they call its `predict(data)` method, passing a Pandas DataFrame matching the model's input schema, which internally calls their `CustomRAGChain.predict()`.
- **C)** It returns a Python dictionary containing the model's configuration (endpoint URL, signature, dependencies) — they must instantiate the model class manually using `CustomRAGChain(**model_dict)` before calling `predict()`.
- **D)** It returns a Flask web server object running on localhost port 5000 — they test the model by sending HTTP POST requests to `http://localhost:5000/invocations` using the `requests` library.

Correct Answer: B **Explanation:** B is correct. `mlflow.pyfunc.load_model()` returns an `mlflow.pyfunc.PyFuncModel` object — a standardized MLflow wrapper that encapsulates the underlying model (whether it's a `PythonModel`, `LangChain` chain, or `sklearn` estimator). This wrapper exposes a uniform `predict(data)` method that accepts a Pandas DataFrame matching the model's input signature. Internally, `PyFuncModel.predict()` calls the developer's `CustomRAGChain.predict()` with the appropriate context. This allows local testing with the same interface that the Model Serving endpoint uses. A is wrong because `load_model()` does NOT return the raw `CustomRAGChain` instance — it returns the `PyFuncModel` wrapper; private methods like `_preprocess()` are not directly accessible on the returned object. C is wrong because `load_model()` returns a ready-to-use model object, not a configuration dictionary; the model is fully instantiated (including `load_context()` having been called). D is wrong because `load_model()` does not start a web server — `mlflow models serve` (a separate CLI command) starts the Flask server for local REST endpoint testing; `load_model()` is for in-process Python testing. **Source:** Section 4: Assembling and Deploying – Objective 1: Code a chain using a pyfunc model — docs.databricks.com (search: "Deploy custom Python model Databricks Model Serving")

Question 45

Difficulty: Beginner

A customer support team wants their employees to interact with a Databricks-hosted chatbot directly from Slack. The chatbot answers questions using a Databricks RAG pipeline. What setup does this require?

- **A)** The Slack administrator installs a custom Slack app that makes HTTP POST requests to the Databricks Model Serving REST endpoint on each user message, using a Service Principal's token stored in Slack's secret manager.
- **B)** The Databricks workspace administrator enables the Databricks Genie app for Slack — admins configure which Genie Agents to expose to Slack, and users interact by mentioning `@Genie` in Slack channels or direct messages.
- **C)** Employees must download the Databricks mobile app and connect it to Slack via the Zapier integration, which bridges Slack messages to Databricks notebook API calls and returns the chatbot's response as a Slack message.
- **D)** The development team deploys a Streamlit app on Databricks Apps with a Slack webhook integration — when users send a Slack message, the webhook triggers the Streamlit app's backend, which calls the chatbot and posts the response to Slack.

Correct Answer: B Explanation: B is correct. The Databricks Genie app for Slack is the purpose-built integration for exposing Databricks conversational agents in Slack. A workspace admin enables the Genie app (available as a Slack app from Databricks), configures which Genie Agents (knowledge bases) are accessible, and pins specific agents to channels. Employees then interact by mentioning `@Genie` (or `@Databricks Genie`) in any Slack channel or DM — asking questions in natural language and receiving answers directly in Slack. A is wrong because while a custom Slack app calling the Model Serving REST endpoint is technically possible, it requires significant custom engineering (Slack app development, webhook handling, token management) — the Databricks Genie Slack app is the purpose-built zero-custom-code solution. C is wrong because there is no "Databricks mobile app to Slack via Zapier" integration — this is not a real Databricks product integration. D is wrong because building a Streamlit-Slack webhook bridge is a complex custom integration that requires maintaining a webhook server — the Genie Slack app provides this out-of-the-box. **Source:** Section 4: Assembling and Deploying – Objective 15: Develop an appropriate interactive user-facing interface — docs.databricks.com (search: "Databricks Genie app Slack Teams")

Question 46

Difficulty: Intermediate

A developer's RAG chain requires the `langchain`, `databricks-vectorsearch`, and `openai` Python packages at serving time. How should these dependencies be specified when logging the model?

- **A)** Include them in a `requirements.txt` file in the same directory as the notebook — the Databricks Model Serving runtime automatically reads `requirements.txt` from the workspace's root directory when deploying any new model version.
- **B)** Pass them as a list to the `pip_requirements` parameter of `mlflow.langchain.log_model()` or `mlflow.pyfunc.log_model()` — MLflow bundles this list with the model artifact, and the serving environment installs these packages when the endpoint starts.
- **C)** Install the packages with `%pip install langchain databricks-vectorsearch openai` in the first cell of the deployment notebook — the serving runtime automatically captures all packages installed in the notebook environment and replicates them.
- **D)** Add the packages to the Databricks cluster configuration's "Advanced Options → Init Script" field — the serving endpoint uses the same cluster configuration as the development cluster, inheriting all installed packages.

Correct Answer: B **Explanation:** B is correct. Dependencies for MLflow models are specified via the `pip_requirements` parameter (or `conda_env` for Conda environments) in `mlflow.*log_model()` calls. These are stored as part of the model artifact (in `requirements.txt` within the MLflow model directory). When a Model Serving endpoint starts, the serving runtime reads these requirements and installs the specified packages in an isolated environment before loading the model. This ensures reproducibility — the serving environment exactly matches the dependencies declared at log time. A is wrong because there is no automatic `requirements.txt` discovery from the workspace root — Model Serving environments are isolated containers that only install what is declared in the model's `pip_requirements` artifact. C is wrong because `%pip install` in a development notebook installs packages in the interactive cluster's environment — this is NOT automatically captured or replicated to the Model Serving environment; serving environments are independent isolated containers. D is wrong because Model Serving endpoints do NOT use the development cluster's configuration — they run on dedicated managed compute with isolated environments defined by the model's logged dependencies. **Source:** Section 4: Assembling and Deploying – Objective 4: Choose basic elements needed to create a RAG application — [docs.databricks.com](https://docs.databricks.com/search/Build%20a%20RAG%20chatbot%20Mosaic%20AI) (search: "Build a RAG chatbot Mosaic AI")

Question 47

Difficulty: Intermediate

A developer adds custom pre-processing to their pyfunc `predict()` method:

```
def predict(self, context, model_input):
    query = model_input["query"].iloc[0]
    if len(query) > 2000:
        query = query[:2000] # truncate long inputs
    response = self.llm.invoke(query)
    return pd.DataFrame({"answer": [response.content]})
```

What potential issue exists with returning `pd.DataFrame({"answer": [response.content]})` instead of a simple string?

- **A)** Returning a Pandas DataFrame from `predict()` is invalid — `mlflow.pyfunc.PythonModel.predict()` must always return a Python dictionary, and returning a DataFrame causes a serialization error in the serving runtime.
- **B)** Returning a single-column DataFrame is valid and the standard pyfunc return format. However, the calling REST API client receives a JSON response with the DataFrame serialized as `{"predictions": [{"answer": "..."}]}` — clients must parse this nested structure to extract the answer string.
- **C)** Returning a Pandas DataFrame causes a performance issue in the serving runtime — DataFrames are serialized to Apache Arrow format before transmission, adding 50–200ms of overhead per request compared to returning a raw string.
- **D)** Returning a Pandas DataFrame works locally but fails in Model Serving because the serving runtime uses Spark to process inference results, and Pandas DataFrames are not compatible with the Spark serialization layer used by Model Serving endpoints.

Correct Answer: B Explanation: B is correct. `mlflow.pyfunc.PythonModel.predict()` can return Pandas DataFrames, and this is a common and valid return format. The serving runtime serializes the DataFrame to JSON in the MLflow inference response format: `{"predictions": [{"answer": "the actual answer text"}]}`. API clients must parse this nested JSON structure to extract the answer. This is important for client developers to know — if they expect a plain string response, they need to handle the nested `predictions[0]["answer"]` path. A is wrong because Pandas DataFrames are a fully supported return type for `predict()` — along with numpy arrays, Python lists, and dictionaries. The standard pyfunc interface explicitly supports multiple return types. C is wrong because while Apache Arrow serialization is used internally, the overhead is measured in milliseconds at most and is not typically a practical concern for most RAG applications where LLM call latency dominates (usually 200ms–2s). D is wrong because Databricks Model Serving runs Python serving code in an isolated Python process, not inside Spark — it does not use Spark serialization for inference responses; Pandas DataFrames work correctly in the serving environment. **Source:** Section 4: Assembling and Deploying – Objective 1: Code a chain using a pyfunc model — [docs.databricks.com \(search: "mlflow.pyfunc PythonModel custom"\)](https://docs.databricks.com/search/mlflow-pyfunc-python-model-custom)

Question 48

Difficulty: Intermediate

A company deploys a RAG application using a Databricks Model Serving endpoint. The security team asks: "When the endpoint calls our internal Vector Search index, whose Unity Catalog credentials are used?" What is the correct answer?

- **A)** The endpoint always uses the `databricks-service-principal@system` anonymous system account for all Unity Catalog data access — individual user identities are not propagated to the serving endpoint's internal resource calls.
- **B)** The endpoint uses the Unity Catalog identity of the user who CREATED the endpoint — this identity is permanently bound to the endpoint at creation time and determines which Unity Catalog resources (Vector Search indexes, Delta tables, UC Functions) the endpoint can access.
- **C)** The endpoint uses the identity of the USER making the inference API call — each incoming request propagates the caller's identity to all downstream Unity Catalog resource access within the serving endpoint.
- **D)** The endpoint uses a rotating service account identity automatically provisioned by Databricks Model Serving — this identity has read-only access to all resources in the workspace and is refreshed every 24 hours for security.

Correct Answer: B **Explanation:** B is correct. This is a critical security concept for Model Serving. The endpoint's Unity Catalog data access identity is permanently bound to the identity of the user (or Service Principal) who CREATED the serving endpoint — not the user making individual inference requests. This "creator identity" is used for all internal resource access: querying Vector Search indexes, reading Delta tables, executing Unity Catalog Functions, accessing Volume files. This is why best practice requires creating production endpoints under a dedicated Service Principal (not a personal user account) — the endpoint's data access capabilities are determined by that Service Principal's Unity Catalog permissions. A is wrong because there is no `databricks-service-principal@system` anonymous account — endpoint access is tied to a specific real identity (the creator). C is wrong because caller identity propagation for data access does NOT occur in Databricks Model Serving — the endpoint uses the creator's identity for internal resource calls, regardless of who is calling the inference API. D is wrong because Databricks does not automatically provision rotating service accounts for Model Serving endpoints — the creator's identity is static (not rotating) and is set permanently at endpoint creation. **Source:** Section 4: Assembling and Deploying – Objective 2: Control access to resources from model serving endpoints — docs.databricks.com (search: "Model serving endpoint permissions Databricks")

Question 49

Difficulty: Intermediate

A developer creates a Vector Search index with `pipeline_type="CONTINUOUS"`. The source Delta table receives updates every 30 seconds from a streaming job. A product manager asks: "How long after a new document is added to the source table will it be searchable?" What is the correct answer?

- **A)** New documents are searchable immediately (0ms latency) — CONTINUOUS sync hooks into Delta Lake's commit log at the transaction level, making vectors available to search before the transaction is even visible to other readers.
- **B)** New documents typically become searchable within seconds to a few minutes — CONTINUOUS sync monitors the Change Data Feed and processes new entries rapidly, but there is inherent pipeline latency for reading CDF, computing embeddings, and updating the ANN index.
- **C)** New documents become searchable within exactly 5 minutes — Databricks guarantees a maximum of 5 minutes sync latency for CONTINUOUS pipeline type, with SLA enforcement via automatic escalation to on-call infrastructure teams.
- **D)** New documents become searchable on the next scheduled sync window — even with CONTINUOUS mode enabled, Databricks batches changes into 15-minute windows to optimize embedding computation costs and ANN index update efficiency.

Correct Answer: B **Explanation:** B is correct. CONTINUOUS sync is near real-time but not zero-latency. The pipeline: (1) The CDF log records the new Delta table transaction. (2) The CONTINUOUS sync pipeline detects the CDF entry (typically within seconds). (3) The new chunk text is sent to the embedding model endpoint for vectorization (adds latency proportional to text length and model throughput). (4) The new vector is added to the ANN index. (5) The vector becomes searchable. Total latency is typically seconds to a few minutes, depending on the embedding model's response time and current processing queue depth. This is called "near real-time" rather than "real-time" because of these inherent pipeline stages. A is wrong because no embedding system can make vectors searchable before the embedding computation is complete — zero-latency would require vectors to be pre-computed before document insertion. C is wrong because there is no Databricks contractual guarantee of exactly 5 minutes for CONTINUOUS sync — it is typically much faster; and "on-call escalation" is not a feature of sync latency SLAs. D is wrong because CONTINUOUS sync does NOT batch into 15-minute windows — that would make it indistinguishable from TRIGGERED sync; the defining characteristic of CONTINUOUS is ongoing, event-driven processing. **Source:** Section 4: Assembling and Deploying – Objective 10: Configure vector search based on requirements — docs.databricks.com (search: "Mosaic AI Vector Search configuration")

Question 50

Difficulty: Intermediate

A developer uses the MLflow Prompt Registry and loads a prompt with

`mlflow.load_prompt("prompts:/rag_system_prompt/production")`. What Python object is returned, and how is it used to assemble a LangChain prompt?

- **A)** It returns a Python string containing the full prompt template text — the string can be directly passed to `ChatPromptTemplate.from_messages([("system", prompt_text)])` to create a LangChain chat prompt.
- **B)** It returns an `mlflow.models.Prompt` object with a `.template` attribute (the prompt text string) and a `.version` attribute (the version number). To use in LangChain: extract `prompt.template` and pass it to `ChatPromptTemplate.from_messages([("system", prompt.template)])`.
- **C)** It returns a LangChain `ChatPromptTemplate` object directly — the MLflow Prompt Registry stores prompts pre-formatted as LangChain objects, enabling direct use in LCEL chains without any conversion step.
- **D)** It returns a callable `PromptFunction` object — calling it with keyword arguments (`prompt_func(context=..., question=...)`) returns the assembled prompt string with variables substituted, without needing to import or use LangChain.

Correct Answer: B Explanation: B is correct. `mlflow.load_prompt()` returns an `mlflow.models.Prompt` object (or `mlflow.prompt.Prompt`). This object contains: (1) `.template` — the raw prompt text string (with placeholder variables like `{context}` and `{question}`). (2) `.version` — the version number of the loaded prompt. (3) `.name` — the registered prompt name. To use in LangChain, the developer extracts the template string and passes it to the appropriate LangChain constructor:

`ChatPromptTemplate.from_messages([("system", prompt.template), ("human", "{question}")])`. A is wrong because `mlflow.load_prompt()` does NOT return a raw string — it returns an `mlflow.models.Prompt` object; accessing the string requires `.template`. C is wrong because the Prompt Registry stores template strings, not framework-specific objects — it is framework-agnostic and stores raw text that developers integrate into any framework (LangChain, LlamaIndex, custom code). D is wrong because there is no `PromptFunction` callable type in the MLflow Prompt Registry — the returned object is a `Prompt` data class with attributes, not a callable function that performs variable substitution. **Source:** Section 4: Assembling and Deploying – Objective 14: Apply prompt version control and manage prompt lifecycle — docs.databricks.com (search: "MLflow Prompt Registry Databricks")

Question 51

Difficulty: Advanced

A developer runs `mlflow.pyfunc.log_model()` but forgets to include `databricks-vectorsearch` in `pip_requirements`. The model works during local testing (because the library is installed in the development cluster) but fails in production. What error occurs, and how should it be fixed without re-running the training pipeline?

- **A)** The model fails with `ImportError: No module named 'databricks.vector_search'` when the serving endpoint tries to load the model's dependencies. Fix: use `mlflow.models.add_pip_requirements(model_uri, ["databricks-vectorsearch"])` to append the missing package without re-logging.
- **B)** The model fails with a `ModelSignatureError` because `databricks-vectorsearch` is part of the model signature definition and its absence causes schema validation to fail. Fix: re-log the model with `pip_requirements` corrected and `input_example` included.
- **C)** The model fails silently — the Vector Search calls are skipped if `databricks-vectorsearch` is not installed, and the serving endpoint falls back to keyword-based search using the source Delta table directly, degrading retrieval quality.
- **D)** The model fails with a `DatabricksVersionError: vectorsearch requires Databricks Runtime 15.0+` because the serving runtime defaults to an older runtime version. Fix: set the endpoint's runtime version to 15.0+ in the endpoint configuration YAML.

Correct Answer: A **Explanation:** A is correct. When a Python package listed in `load_context()` or `predict()` is imported but missing from `pip_requirements`, the Model Serving endpoint fails with `ImportError` when the serving container tries to load the model. The fix WITHOUT re-running the full training pipeline: use `mlflow.models.add_pip_requirements(model_uri="runs:/<run_id>/model", requirements=["databricks-vectorsearch==0.x.y"])` to append the missing package to the already-logged model artifact's `requirements.txt`. This modifies only the dependency specification without touching the model weights, code, or signature. Alternatively, use `mlflow.pyfunc.get_model_dependencies(model_uri)` to inspect what was originally logged. B is wrong because `databricks-vectorsearch` is a Python runtime library, not a model signature component — its absence causes an `ImportError`, not a `ModelSignatureError`. C is wrong because Python does not "silently skip" missing imports — attempting to import a missing module raises `ImportError` immediately; there is no fallback to keyword search. D is wrong because `databricks-vectorsearch` library compatibility is unrelated to Databricks Runtime version in this context — the error is a missing package, not a runtime version incompatibility. **Source:** Section 4: Assembling and Deploying – Objective 4: Choose basic elements needed to create a RAG application — docs.databricks.com (search: "Build a RAG chatbot Mosaic AI" and "mlflow.pyfunc PythonModel custom")

Question 52

Difficulty: Advanced

A team implements a CI/CD pipeline with three environments: dev, staging, production. The MLflow Prompt Registry has a prompt `main.ml.support_prompt` with: version 1 (alias: `dev`), version 2 (alias: `staging`), version 3 (alias: `production`). The application loads with

`mlflow.load_prompt("prompts:/main.ml.support_prompt/production")`. After a successful staging evaluation, the team wants to promote version 2 to production without any code changes. What is the exact sequence of actions?

- **A)** (1) Delete the `production` alias from version 3 → (2) Assign the `production` alias to version 2 → (3) Application automatically loads version 2 on the next inference request without endpoint redeployment.
- **B)** (1) Register a new version 4 that is a copy of version 2 → (2) Assign `production` alias to version 4 → (3) Redeploy the Model Serving endpoint to pick up the new prompt version → (4) Application loads version 4 (the copy of the staging prompt).
- **C)** (1) Re-assign the `production` alias to point to version 2 in the MLflow Prompt Registry UI or via `mlflow.client.MlflowClient().set_registered_model_alias(name="main.ml.support_prompt", alias="production", version=2)` → (2) The application loads version 2 on the next call without any code or endpoint changes.
- **D)** (1) Export version 2's prompt text to a text file → (2) Import it as a new version in the `production` catalog schema → (3) Update the application code to load from the new version URI → (4) Redeploy the serving endpoint.

Correct Answer: C **Explanation:** C is correct. The alias-based promotion workflow is the core value of MLflow Prompt Registry: (1) The team re-assigns the `production` alias to version 2 (previously on version 3). This can be done via the MLflow UI or `MlflowClient().set_registered_model_alias(name="main.ml.support_prompt", alias="production", version=2)`. (2) The application code

`mlflow.load_prompt("prompts:/main.ml.support_prompt/production")` always resolves the `production` alias at LOAD TIME — on the next inference request, it fetches the current target of the `production` alias, which is now version 2. (3) No code changes, no model re-logging, no endpoint redeployment needed — the prompt update is zero-downtime and instant. A is wrong because deleting the `production` alias from version 3 before assigning it to version 2 would create a brief window where `production` alias is unassigned — the application would throw a `PromptVersionNotFound` error. The correct order is to assign first (MLflow reassigns atomically).

B is wrong because creating a copy as version 4 is unnecessary overhead — and requiring an endpoint redeployment defeats the purpose of alias-based promotion (which specifically avoids redeployment). D is wrong because alias-based promotion was invented precisely to eliminate this export/reimport/redeploy workflow. **Source:** Section 4: Assembling and Deploying – Objective 14: Apply prompt version control and manage prompt lifecycle — docs.databricks.com (search: "MLflow Prompt Registry Databricks" and "MLflow Prompt Registry aliases")

Question 53

Difficulty: Advanced

A team develops a multi-step RAG agent where the `predict()` method of their pyfunc model calls Vector Search, then an LLM, then applies post-processing. They want to test only the post-processing step in isolation. Which testing approach is most correct?

- **A)** Deploy the full model to a staging endpoint and send carefully crafted test inputs that bypass the Vector Search and LLM steps by including `skip_retrieval=true` in the request payload — a flag the `predict()` method checks to short-circuit earlier steps.
- **B)** Extract the `_postprocess()` method (or equivalent logic) from the pyfunc class and test it independently as a pure Python function using `pytest` — unit testing post-processing logic requires only Python inputs and assertions, not any MLflow or Databricks infrastructure.
- **C)** Use `mlflow.pyfunc.load_model()` to load the full model and call `predict()` with inputs that are pre-formatted to skip pre-processing — MLflow's pyfunc interface automatically detects formatted inputs and routes them directly to `_postprocess()`.
- **D)** Create a separate MLflow run with a pyfunc model that contains only the `_postprocess()` method, log it independently, deploy to a separate serving endpoint, and test through that endpoint — modular endpoint testing is the standard approach for component isolation.

Correct Answer: B Explanation: B is correct. Post-processing logic in a pyfunc model (like JSON parsing, confidence filtering, citation formatting) is pure Python business logic that takes some input and returns a transformed output. The correct, efficient testing approach is to extract and test this logic as an independent Python function: `from my_module import postprocess_response; result = postprocess_response(sample_llm_output); assert result["confidence"] > 0.7`. This runs in milliseconds without any infrastructure, making it fast and suitable for CI. A is wrong because adding `skip_retrieval=true` request-level flags is an anti-pattern — the production code should not include test-bypass logic; it adds maintenance overhead and can mask real integration issues. C is wrong because MLflow pyfunc `predict()` does not have built-in routing based on input formatting — it calls the full `predict()` method as implemented; MLflow provides no "automatic step skipping" feature. D is wrong because deploying a separate endpoint just to test a post-processing function is vastly over-engineered — post-processing is pure Python and needs no serving infrastructure for unit testing. **Source:** Section 4: Assembling and Deploying – Objective 12: Apply CI/CD best practices — docs.databricks.com (search: "Databricks Asset Bundles CI/CD")

Question 54

Difficulty: Advanced

A developer creates a RAG chain where the input is a list of user messages in OpenAI chat format:

```
{"messages": [{"role": "user", "content": "What is the return policy?"}]}
```

They want to log this with the correct MLflow signature. Which code correctly defines the signature?

- **A)** `python from mlflow.models import infer_signature import pandas as pd input_example = pd.DataFrame([{"messages": [{"role": "user", "content": "test"}]}) signature = infer_signature(input_example, pd.DataFrame([{"response": "test answer"}]))`

```
* **B)** python
from mlflow.models.signature import ModelSignature
from mlflow.types.schema import Schema, ColSpec
signature = ModelSignature(
    inputs=Schema([ColSpec("string", "messages")]),
    outputs=Schema([ColSpec("string", "response")])
)
```

- **C)** `python from mlflow.models import infer_signature input_example = {"messages": [{"role": "user", "content": "What is the return policy?"}]} output_example = {"response": "Our return policy is 30 days."} signature = infer_signature(model_input=input_example, model_output=output_example)`

```
* **D)** python
import mlflow
mlflow.langchain.log_model(
    lc_model=chain,
    artifact_path="chain",
    input_example={"messages": [{"role": "user", "content": "What is the return policy?"}]}
)
# MLflow auto-infers the correct complex schema from the input_example
```

Correct Answer: D **Explanation:** D is correct and is the recommended approach for LangChain models. When `input_example` is provided to `mlflow.langchain.log_model()`, MLflow 2.5+ automatically inspects the example, infers the complete schema (including nested list-of-dicts structures like the OpenAI messages format), and creates the model signature automatically. This handles complex nested schemas (lists, dicts, lists of dicts) that manual signature definition is error-prone for. The developer doesn't need to manually construct the signature. A is partially correct (provides an `input_example` to `infer_signature`) but incorrectly serializes the messages list as a JSON string in a DataFrame column — this changes the schema from nested objects to a flat string, which doesn't match the actual input format. B is wrong because `ColSpec("string", "messages")` defines `messages` as a single string column, not as a list of message objects — this schema mismatch would cause payload validation failures for structured chat inputs. C is wrong because `infer_signature()` with raw dictionaries (not DataFrames) may not correctly infer the schema for list-of-dict inputs — `infer_signature` requires Pandas DataFrames or numpy arrays for reliable schema inference. D is correct and also the simplest approach. **Source:** Section 4: Assembling and Deploying – Objective 4 & 5: RAG elements and Unity Catalog

Question 55

Difficulty: Advanced

A data engineer is implementing a batch inference pipeline using `ai_query()` to extract structured JSON from 1 million contract documents. After the pipeline runs, the Query Profile shows 150,000 rows have `null` in the `extracted_json` column. What should they investigate, and what remediation applies?

- **A)** The 150,000 nulls indicate that the Model Serving endpoint rate limit was hit for those rows — they should rerun with a smaller batch size (e.g., process 850,000 rows first, then 150,000 in a second run) to stay within rate limits.
- **B)** The 150,000 nulls typically indicate either: (a) the LLM failed to extract valid JSON for those contracts (model error — investigate contract characteristics for those rows), (b) those contracts exceeded the context window limit (investigate average text length for null rows), or (c) timeout errors for very long contracts. Remediation: filter for null rows, investigate patterns (text length, contract type), and apply targeted pre-processing (truncation, summarization) before re-running inference on the failed rows.
- **C)** The 150,000 nulls are expected — `ai_query()` returns null for all rows where the model returns an answer with confidence below 0.7, and the developer should tune the confidence threshold parameter to reduce null outputs.
- **D)** The 150,000 nulls indicate that the Serverless SQL Warehouse ran out of memory — rerun the query on a Classic SQL Warehouse with at least 32GB of driver memory to handle the full 1 million row batch without OOM errors.

Correct Answer: B **Explanation:** B is correct. `ai_query()` returns `null` when the model call fails for a specific row, which can happen due to multiple causes: (1) **Model error** — the LLM could not produce valid JSON for that contract's content (malformed prompts, unusual content structure). (2) **Context window exceeded** — contracts longer than the model's context window cause the inference call to fail. (3) **Timeout** — very long contracts take too long to process and the request times out. The correct investigation approach: `SELECT id, LENGTH(contract_text), extracted_json IS NULL AS failed FROM results WHERE extracted_json IS NULL` — check if null rows correlate with long text (context window issue) or certain contract types (model failure pattern). A is wrong because rate limit failures in `ai_query()` are handled internally with automatic retries — they would appear as delays or partial null patterns, not as a consistent 15% failure rate; and rerunning a "smaller batch" doesn't fix rate limits. C is wrong because `ai_query()` has no built-in "confidence threshold" parameter — null results indicate inference failures, not low-confidence outputs. D is wrong because `ai_query()` runs on Serverless SQL Warehouses, which auto-scale and do not have fixed memory limits like Classic warehouses; switching to Classic would not help and would actually break `ai_query()` (which requires Serverless). **Source:** Section 4: Assembling and Deploying – Objective 9: Identify batch inference workloads and apply `ai_query()` — docs.databricks.com (search: "ai_query function Databricks SQL" and "Batch inference AI functions")

Question 56

Difficulty: Advanced

A team's custom MCP server (hosted on Databricks Apps) exposes a `query_customer_database()` tool that searches a confidential customer PII database. A security audit requires: (1) all calls to this tool must be logged for compliance, (2) only users with `customer_data_analyst` role can invoke this tool, (3) rate limiting of 100 calls/minute per user must be enforced. Which Databricks component provides all three capabilities?

- **A)** Databricks Secrets — store the customer database credentials as secrets, and implement logging, authorization, and rate limiting as custom Python middleware within the Databricks App's FastAPI code.
- **B)** Unity AI Gateway — all MCP server traffic routes through the Unity AI Gateway, which provides: (1) centralized audit logging of all tool invocations, (2) identity-based access control (only `customer_data_analyst` role can call the tool), and (3) configurable rate limits per user or group.
- **C)** Databricks Workflows — configure a Workflow to run a pre-invocation security check Task before the MCP server tool is called, and a post-invocation logging Task after — Workflows provide sequential task orchestration with built-in retry and audit trail.
- **D)** Unity Catalog row-level security — apply a row filter on the customer PII table that only returns rows when the calling user has `customer_data_analyst` role, and rely on Unity Catalog's query history for audit logging and its connection management for rate limiting.

Correct Answer: B **Explanation:** B is correct. The Unity AI Gateway is the central governance layer for all MCP server traffic in Databricks. It provides exactly the three required capabilities: (1) **Audit logging** — all tool invocations through the Gateway are logged with caller identity, timestamp, tool name, and request/response metadata. (2) **Access control** — Unity AI Gateway policies can enforce identity-based authorization, allowing only users or service principals with specific roles (like `customer_data_analyst`) to invoke specific tools. (3) **Rate limiting** — configurable per-user or per-group rate limits can be set directly in the Gateway configuration. The Gateway intercepts all MCP traffic at the protocol level, enforcing these policies before the request reaches the Databricks App. A is wrong because implementing logging, authorization, and rate limiting as custom Python middleware in the app code is fragile, harder to audit centrally, and bypassed if the MCP server is called directly without going through the middleware. C is wrong because Databricks Workflows are batch/scheduled job orchestrators — they cannot intercept and gate individual real-time MCP tool calls; security policies for real-time API calls require the Unity AI Gateway. D is wrong because Unity Catalog row-level security governs data access at the table query level — it doesn't provide tool-invocation-level rate limiting or structured audit trails for MCP protocol calls. **Source:** Section 4: Assembling and Deploying – Objective 13: Integrate managed, external, and custom MCP servers — docs.databricks.com (search: "MCP servers Databricks" and "Unity Catalog MCP services")

Question 57

Difficulty: Proficiency

A platform engineer is designing the serving infrastructure for a RAG chatbot with these non-functional requirements: (1) 99.9% availability SLA, (2) P99 latency < 300ms, (3) handles 500 concurrent users with spiky traffic (0 to 500 in 30 seconds), (4) new model versions must be deployable in < 5 minutes without downtime. Evaluate whether Provisioned Throughput or pay-per-token endpoints better satisfy each requirement.

- **A)** Pay-per-token satisfies all four requirements better — it scales from zero to 500 concurrent users instantly (serverless auto-scaling), has no fixed capacity constraints, deploys in < 5 minutes, and Databricks guarantees 99.9% availability for all Foundation Model API endpoints.
- **B)** Provisioned Throughput satisfies (1) availability and (4) deployment speed (< 5 minutes via API update), but struggles with (3) spiky traffic (requires over-provisioning for peak load since capacity is fixed) and may or may not meet (2) latency (< 300ms achievable at correct TPS provisioning level). Pay-per-token handles (3) spiky traffic better but offers no P99 latency guarantee for (2) and has shared infrastructure availability risk for (1).
- **C)** Provisioned Throughput satisfies all four requirements — it provides guaranteed capacity (99.9% availability), guaranteed P99 latency (predictable at provisioned TPS), absorbs the 0-to-500 spike through pre-provisioned capacity headroom, and supports < 5-minute hot-swap deployments.
- **D)** Neither endpoint type satisfies all requirements — a hybrid architecture using Provisioned Throughput as a baseline (for low-latency guaranteed requests) with pay-per-token overflow (for traffic spikes) achieves all four requirements simultaneously.

Correct Answer: D **Explanation:** D is correct. Neither endpoint type perfectly satisfies all four requirements alone — a hybrid is optimal: (1) **99.9% availability** → Both types are managed by Databricks. Provisioned Throughput provides more predictable availability since dedicated compute is reserved. A hybrid retains availability via failover. (2) **P99 < 300ms** → Provisioned Throughput with sufficient TPS provisioning can guarantee P99 latency. Pay-per-token has no latency SLA and may spike under load. (3) **0 to 500 in 30 seconds** → Pay-per-token handles spikes (serverless auto-scaling). Provisioned Throughput requires over-provisioning for 500-user peak, which is expensive at 0-user off-peak. A hybrid uses Provisioned Throughput for baseline (e.g., 200 users) and overflows to pay-per-token for spikes (201–500 users). (4) **< 5 min downtime-free deployment** → Both support traffic splitting for zero-downtime deployment; update time is comparable. A is wrong because pay-per-token provides NO P99 latency guarantee — under load, latency can spike significantly. C is wrong because Provisioned Throughput has FIXED capacity — a sudden 0-to-500 spike in 30 seconds with under-provisioned Provisioned Throughput results in throttling and latency degradation; it requires over-provisioning which is costly at low utilization. B is partially correct but doesn't identify the hybrid as the solution. D captures the full nuance. **Source:** Section 4: Assembling and Deploying – Objective 7: Identify how to serve an LLM application — docs.databricks.com (search: "Databricks Foundation Model APIs" and "Provisioned throughput model serving")

Question 58

Difficulty: Proficiency

A team's production RAG application has the following architecture: Databricks Apps (Streamlit front-end) → Model Serving endpoint (pyfunc chain) → Vector Search index (Delta Sync, CONTINUOUS) → Foundation Model API. After 3 months, they need to update ALL of the following: (1) the system prompt (improved instructions), (2) the embedding model (from `bge-small-en` to `bge-large-en` for better quality), (3) the pyfunc chain code (new output parser). List the correct deployment sequence and the risk at each step.

- **A)** (1) Update the prompt in MLflow Prompt Registry → re-assign `production` alias → no downtime. (2) Update embedding model → requires re-embedding ALL documents → rebuild entire Vector Search index (significant downtime risk if not managed carefully). (3) Update pyfunc chain → log new model version → traffic split from old to new version → zero downtime.
- **B)** (1) Redeploy the Databricks App with new prompt hardcoded in Streamlit code → no redeployment of Model Serving endpoint needed. (2) Replace `bge-small-en` with `bge-large-en` in the existing Vector Search index configuration → the index automatically re-embeds all documents in background. (3) Push new pyfunc code to Git → MLflow auto-deploys from Git.
- **C)** All three changes can be deployed atomically by creating a new branch in Git, updating all files, and running `databricks bundle deploy` → this deploys all changes simultaneously with zero downtime because Asset Bundles apply changes transactionally.
- **D)** (1) Update the embedding model FIRST (most impactful) → re-embed all documents → sync new index. Then (2) update pyfunc chain code → deploy. Then (3) update prompt → re-assign alias. The prompt update should always be last because prompts can cause unexpected behavior that masks embedding model issues.

Correct Answer: A **Explanation:** A is correct. The deployment sequence and risks: (1) **Prompt update** → MLflow Prompt Registry alias re-assignment is instant and zero-downtime — application immediately loads the new prompt on the next inference call. Risk: minimal; can be instantly rolled back by re-assigning the alias to the previous version. (2) **Embedding model update** → This is the most operationally complex step. Changing the embedding model requires: (a) deleting the old index or creating a new index with the new model configuration, (b) re-embedding ALL 500K+ documents with the new model, (c) rebuilding the ANN index from scratch. During re-embedding, if the old index is deleted, retrieval fails. Mitigation: create a NEW index with the new model, let it sync fully, update the chain code to point to the new index, traffic split, then delete the old index. Risk: HIGH — downtime risk if not managed with parallel index approach. (3) **Pyfunc chain update** → log new version to Unity Catalog → traffic split (10% → 100%) → zero downtime. Risk: low with canary deployment. B is wrong because embedding model configuration on an existing index cannot simply be swapped; re-indexing is required. C is wrong because `databricks bundle deploy` applies changes to infrastructure resources but does NOT handle the data migration required for re-embedding. D's sequencing is partially valid but the reasoning about "prompt always last" is arbitrary business logic, not a technical requirement. **Source:** Section 4: Assembling and Deploying – Objectives 5, 8, 12, 14 — docs.databricks.com (search: "Register model Unity Catalog MLflow" and "Create Mosaic AI Vector Search index" and "MLflow Prompt Registry Databricks")

Question 59

Difficulty: Proficiency

A developer designs a Managed Agent Memory integration for a customer service agent. Customers return multiple times over days/weeks. Requirements: (1) remember each customer's name and account tier (persists across sessions), (2) remember the issues discussed in the current support session (within session only), (3) all memory must be governed by Unity Catalog access controls (no customer A can read customer B's memory). Which Databricks architecture satisfies all three requirements?

- **A)** Use Managed Agent Memory for (1) cross-session preferences (Unity Catalog-governed, per-user namespaced), LangGraph Checkpointer with Lakebase for (2) within-session conversation history (`thread_id = session_id`), and (3) Unity Catalog row-level security applied to the Lakebase memory table (filtering rows by `customer_id`).
- **B)** Use a single Redis cache for all memory — Redis supports both session-scoped TTL keys (for within-session memory) and persistent keys (for cross-session memory), and its Redis ACL feature provides access isolation between customers.
- **C)** Use MLflow Experiment tags for (1) cross-session data (stored as run parameters on the customer's dedicated experiment), and Unity Catalog Delta Lake for (2) session data (each session writes a new row), with (3) access controlled by Unity Catalog table permissions.
- **D)** Use Inference Tables for both (1) and (2) — Inference Tables capture all inputs/outputs and the agent can query the table at session start to reconstruct previous context, with Unity Catalog permissions on the table providing customer isolation.

Correct Answer: A **Explanation:** A is correct. This architecture maps each requirement to the appropriate Databricks component: (1) **Cross-session memory (name, account tier)** → Managed Agent Memory is the purpose-built Databricks service for this — it is Unity Catalog-governed, supports per-user namespaced key-value storage (no customer A can read customer B's namespace), and is fully managed with zero infrastructure. (2) **Within-session conversation history** → LangGraph Checkpointer with Lakebase stores conversation state within a `thread_id` (= current session ID). When the session ends, the thread's checkpoint remains in Lakebase but is not automatically loaded in the next session. (3) **Unity Catalog governance** → Managed Agent Memory uses UC governance natively. The Lakebase session memory table can have row-level security policies (`WHERE customer_id = current_user_customer_id`) applied via Unity Catalog. B is wrong because Redis is an external SaaS dependency outside the Databricks security perimeter — it doesn't provide Unity Catalog governance and requires separate infrastructure management. C is wrong because MLflow Experiment tags are for ML experiment metadata tracking — using them as a per-customer cross-session memory store is an anti-pattern that scales poorly and lacks proper identity-based access controls. D is wrong because Inference Tables are monitoring tools (logging production traffic) — using them as a memory retrieval system by querying past rows creates high latency at session start and does not provide the structured, indexed memory access pattern needed for production agent memory. **Source:** Section 4: Assembling and Deploying – Objective 11: Configure a persistent datastore — docs.databricks.com (search: "Agent memory Databricks" and "Lakebase Databricks Postgres")

Question 60

Difficulty: Proficiency

A senior engineer reviews a junior developer's implementation of a Vector Search index creation for a real-time news chatbot (knowledge base updated continuously every 30 seconds with breaking news). The junior developer uses:

```
vsc.create_delta_sync_index(  
    endpoint_name="news_vs_endpoint",  
    source_table_name="main.news.articles_chunks",  
    index_name="main.news.articles_index",  
    pipeline_type="TRIGGERED",  
    primary_key="chunk_id",  
    embedding_source_column="chunk_text",  
    embedding_model_endpoint_name="databricks-bge-small-en"  
)
```

Identify ALL issues with this configuration for the stated use case and explain the correct configuration.

- **A)** The only issue is the embedding model — `bge-small-en` should be replaced with `bge-large-en` for news content, which has more diverse vocabulary than general text. The `TRIGGERED` pipeline type is acceptable for a 30-second update frequency.
- **B)** Three issues: (1) `pipeline_type="TRIGGERED"` should be `"CONTINUOUS"` — news updates every 30 seconds require near-real-time sync, not manual triggers. (2) `databricks-bge-small-en` may be underpowered for the diverse vocabulary of breaking news — `bge-large-en` provides better semantic coverage. (3) The Vector Search endpoint size `news_vs_endpoint` should be verified to handle the high write throughput from continuous news ingestion, which requires adequate endpoint provisioning.
- **C)** The only issue is the `primary_key` — for news chunks that update frequently, the primary key should be a composite key of `(article_id, chunk_index)` to uniquely identify each chunk within an article; using only `chunk_id` causes update conflicts when existing chunks are modified.
- **D)** There are no issues — `TRIGGERED` with a 30-second manual trigger schedule is equivalent to `CONTINUOUS` mode for practical purposes, and `bge-small-en` is recommended for high-throughput real-time pipelines because its smaller size reduces embedding latency.

Correct Answer: B **Explanation:** B is correct. Multiple issues exist: (1) `pipeline_type="TRIGGERED"` is critically wrong for this use case. News updates every 30 seconds — `TRIGGERED` mode only syncs when explicitly triggered by calling `index.sync()`. Someone must call `sync()` every 30 seconds via a scheduled job, adding orchestration complexity. More importantly, each sync call itself takes time to process — if the sync takes longer than 30 seconds, articles are permanently delayed. `CONTINUOUS` mode is the correct choice: it monitors CDF continuously and processes changes as they arrive, achieving near-real-time freshness without manual orchestration. (2) `databricks-bge-small-en` may be insufficient — `bge-small-en` (33M parameters) is optimized for speed over accuracy. For news content with diverse, rapidly evolving vocabulary (breaking events, new names, technical jargon), `bge-large-en` (335M parameters) provides significantly better semantic representation for retrieval quality. (3) **Endpoint provisioning** — continuous high-frequency ingestion from a real-time news pipeline requires adequate Vector Search endpoint compute; an undersized endpoint creates a bottleneck. A is wrong because it accepts `TRIGGERED` as appropriate — for a 30-second update cycle, `TRIGGERED` is operationally incorrect. D is wrong because `TRIGGERED` and `CONTINUOUS` are NOT equivalent —

TRIGGERED requires explicit API calls to trigger sync, cannot react automatically to new data, and has higher operational overhead. **Source:** Section 4: Assembling and Deploying – Objectives 8 & 10: Key concepts and configuration of Mosaic AI Vector Search — docs.databricks.com (search: "Mosaic AI Vector Search overview" and "Mosaic AI Vector Search configuration")